MAID 3.1

Revision 20

March 13, 2026

Contents

1 Authors 7

2 Usage 8

2.1 Module Loading 8
2.2 Module Initialization 8
2.3 Device Connection 9
2.4 Capability Enumeration 9
2.5 Shooting Still Images (Single Shooting) 11
2.6 Data Transfer 11
2.7 Disconnect Device 11
2.8 Module Termination 11

3 Enumerations 13

3.1 Result Codes 14
3.2 Data Object Types 15
3.3 Data Types 15
3.4 Array Types 16
3.5 Capability Types 16
3.6 Capability Operations 16
3.7 Capability Visibility 17
3.8 Object Types 17
3.9 Events 17
3.10 User Interface Request Types 18
3.11 User Interface Results 18
3.12 Filters 18
3.13 Commands 19
3.14 Capabilities 20
3.15 Color Spaces 22
3.16 Boolean Defaults 22
3.17 Module Types 23
3.18 File Data Types 23
3.19 Flash Modes 23
3.20 Fmm Manual Setting 23
3.21 F0 Manual Setting 24
3.22 LV Status 25

4 Structures and Types 27

4.1 Word Value 27
4.2 Unsigned Long Value 27
4.3 Parameter Value 27
4.4 Pointer Value 27
4.5 Reference Value 27
4.6 MAID Entry Point Function Pointer 27
4.7 MAID Completion Function Pointer 28
4.8 MAID Data Delivery Function Pointer 28
4.9 MAID Event Notification Function Pointer 28
4.10 MAID Progress Notification Function Pointer 28

4.11 MAID User Interface Request Function Pointer 28
4.12 Callback Definition Structure 28
4.13 Date/Time Structure 28
4.14 Point Structure 29
4.15 Size Structure 29
4.16 Rectangle Structure 29
4.17 String Structure 29
4.18 Array Structure 29
4.19 Range Structure 30
4.20 Capability Information Structure 30
4.21 Object Structure 31
4.22 User Interface Request Structure 31
4.23 Generic Data Delivery Structure 32
4.24 Image Data Delivery Structure 32
4.25 Sound Data Delivery Structure 33
4.26 Enumeration Structure 34
4.27 File Data Delivery Structure 34
4.28 LiveView Header Structure 35
4.29 LiveView Data Delivery Structure 36
4.30 Callback NkMAIDCSCallback Structure 36

5 Result Codes 37

5.1 kNkMAIDResult_NotSupported 37
5.2 kNkMAIDResult_UnexpectedDataType 37
5.3 kNkMAIDResult_ValueOutOfBounds 37
5.4 kNkMAIDResult_BufferSize 37
5.5 kNkMAIDResult_Aborted 37
5.6 kNkMAIDResult_NoMedia 37
5.7 kNkMAIDResult_NoEventProc 37
5.8 kNkMAIDResult_NoDataProc 37
5.9 kNkMAIDResult_ZombieObject 38
5.10 kNkMAIDResult_OutOfMemory 38
5.11 kNkMAIDResult_UnexpectedError 38
5.12 kNkMAIDResult_HardwareError 38
5.13 kNkMAIDResult_MissingComponent 38
5.14 kNkMAIDAPIResult_DeviceNotAvailable 38
5.15 kNkMAIDAPIResult_OpenSessionFailed 38
5.16 kNkMAIDAPIResult_LiveViewAlreadyStarted 38
5.17 kNkMAIDAPIResult_LiveViewAlreadyStopped 38
5.18 kNkMAIDAPIResult_InvalidDeviceID 38
5.19 kNkMAIDAPIResult_StartLiveViewFailed 38
5.20 kNkMAIDAPIResult_DeviceFailed 38
5.21 kNkMAIDAPIResult_OpenSessionFailed 39
5.22 kNkMAIDAPIResult_CapabilityNotSupported 39
5.23 kNkMAIDAPIResult_OperationNotSupported 39
5.24 kNkMAIDAPIResult_InvalidCapID 39
5.25 kNkMAIDAPIResult_OutOfRangeValue 39
5.26 kNkMAIDAPIResult_InvalidShootingType 39
5.27 kNkMAIDAPIResult_ShootingCancelFailed 39
5.28 kNkMAIDAPIResult_InvalidShootCount 39
5.29 kNkMAIDAPIResult_InvalidShootingMode 39
5.30 kNkMAIDAPIResult_InvalidFileSavePath 39
5.31 kNkMAIDAPIResult_MovieStartFailed 39
5.32 kNkMAIDAPIResult_FileSaveFailed 39
5.33 kNkMAIDAPIResult_InvalidInterval 40
5.34 kNkMAIDAPIResult_InvalidStartTime 40

5.35 kNkMAIDAPIResult_InvalidBULBExposureTime 40
5.36 kNkMAIDAPIResult_InvalidArguments 40
5.37 kNkMAIDAPIResult_InvalidState 40
5.38 kNkMAIDAPIResult_ContinousShootIntrupted 40
5.39 kNkMAIDResult_NoError 40
5.40 kNkMAIDResult_Pending 40
5.41 kNkMAIDResult_OrphanedChildren 40
5.42 kNkMAIDResult_VendorBase 40

6 Events 41
6.1 kNkMAIDEvent_AddChild 41
6.2 kNkMAIDEvent_RemoveChild 41
6.3 kNkMAIDEvent_WarmingUp 41
6.4 kNkMAIDEvent_WarmedUp 41
6.5 kNkMAIDEvent_CapChange 41
6.6 kNkMAIDEvent_OrphanedChildren 42
6.7 kNkMAIDEvent_CapChangeValueOnly 42
6.8 kNkMAIDEvent_DeviceInfoChanged 42
6.9 kNkMAIDEvent_ImageSaved 42
6.10 kNkMAIDEvent_ImageSavedInCard 42
6.11 kNkMAIDEvent_StorageFull_ImageNotSaved 42
6.12 kNkMAIDEvent_AcquireFailed_ImageNotSaved 43
6.13 kNkMAIDEvent_ChangeSaveDestination 43
6.12 kNkMAIDEvent_AddImageInCard 43

7 Commands 45
7.1 kNkMAIDCommand_Async 45
7.2 kNkMAIDCommand_Open 46
7.3 kNkMAIDCommand_Close 46
7.4 kNkMAIDCommand_GetCapCount 47
7.5 kNkMAIDCommand_GetCapInfo 47
7.6 kNkMAIDCommand_CapStart 47
7.7 kNkMAIDCommand_CapSet 47
7.8 kNkMAIDCommand_CapGet 48
7.9 kNkMAIDCommand_CapGetDefault 49
7.10 kNkMAIDCommand_CapGetArray 49
7.11 kNkMAIDCommand_Mark 50
7.12 kNkMAIDCommand_AbortToMark 50
7.13 kNkMAIDCommand_Abort 50
7.14 kNkMAIDCommand_EnumChildren 51
7.15 kNkMAIDCommand_GetParent 51
7.16 kNkMAIDCommand_ResetToDefault 51

8 Capabilities 53
8.1 kNkMAIDCapability_AsyncRate 53
8.2 kNkMAIDCapability_ProgressProc 54
8.3 kNkMAIDCapability_EventProc 54
8.4 kNkMAIDCapability_DataProc 54
8.5 kNkMAIDCapability_UIRequestProc 54
8.6 kNkMAIDCapability_IsAlive 55
8.7 kNkMAIDCapability_Children 55
8.8 kNkMAIDCapability_State 55
8.9 kNkMAIDCapability_Name 55
8.10 kNkMAIDCapability_Description 56
8.11 kNkMAIDCapability_Interface 56
8.12 kNkMAIDCapability_DataTypes 56

8.13 kNkMAIDCapability_DateTime 56
8.14 kNkMAIDCapability_StoredBytes 56
8.15 kNkMAIDCapability_Eject 57
8.16 kNkMAIDCapability_Feed 57
8.17 kNkMAIDCapability_Capture 57
8.18 kNkMAIDCapability_MediaPresent 57
8.19 kNkMAIDCapability_Mode 57
8.20 kNkMAIDCapability_Acquire 58
8.21 kNkMAIDCapability_ForceScan 58
8.22 kNkMAIDCapability_Start 58
8.23 kNkMAIDCapability_Length 58
8.24 kNkMAIDCapability_SampleRate 58
8.25 kNkMAIDCapability_Stereo 59
8.26 kNkMAIDCapability_Samples 59
8.27 kNkMAIDCapability_Filter 59
8.28 kNkMAIDCapability_Prescan 59
8.29 kNkMAIDCapability_ForcePrescan 59
8.30 kNkMAIDCapability_ForceAutoFocus 60
8.31 kNkMAIDCapability_AutoFocus 60
8.32 kNkMAIDCapability_AutoFocusPt 60
8.33 kNkMAIDCapability_Focus 60
8.34 kNkMAIDCapability_Coords 60
8.35 kNkMAIDCapability_Resolution 61
8.36 kNkMAIDCapability_Preview 61
8.37 kNkMAIDCapability_Negative 61
8.38 kNkMAIDCapability_ColorSpace 61
8.39 kNkMAIDCapability_Bits 61
8.40 kNkMAIDCapability_Planar 62
8.41 kNkMAIDCapability_Lut 62
8.42 kNkMAIDCapability_Transparency 62
8.43 kNkMAIDCapability_Threshold 62
8.44 kNkMAIDCapability_Pixels 63
8.45 kNkMAIDCapability_NegativeDefault 63
8.46 kNkMAIDCapability_Firmware 63
8.47 kNkMAIDCapability_CommunicationLevel1 63
8.48 kNkMAIDCapability_CommunicationLevel2 64
8.49 kNkMAIDCapability_BatteryLevel 64
8.50 kNkMAIDCapability_FreeBytes 64
8.51 kNkMAIDCapability_FreeItems 64
8.52 kNkMAIDCapability_Remove 65
8.53 kNkMAIDCapability_FlashMode 65
8.54 kNkMAIDCapability_ModuleType 65
8.55 kNkMAIDCapability_AcquireStreamStart 65
8.56 kNkMAIDCapability_AcquireStreamStop 65
8.57 kNkMAIDCapability_AcceptDiskAcquisition 66
8.58 kNkMAIDCapability_Version 66
8.59 kNkMAIDCapability_FilmFormat 66
8.60 kNkMAIDCapability_TotalBytes 67

9 Function Definitions 69

9.1 MAID Entry Point Function 69
9.2 MAID Completion Function 69
9.3 MAID Data Delivery Function 69
9.4 MAID Event Notification Function 69
9.5 MAID Progress Notification Function 70
9.6 MAID User Interface Request Function 70

9.7 LiveViewData Function 70
9.8 MAID EntryPointE Function 71
9.9 MAID SetHinstance Function 71
9.10 MAID AllocateMemory Function 71
9.11 MAID FreeMemory Function 71
9.12 MAID AsyncCompletion Function 72
9.13 MAID InitializeSDK Function 72
9.14 MAID FreeSDK Function 74
9.15 MAID ConnectDevice Function 74
9.16 MAID DisconnectDevice Function 75
9.17 MAID StartLiveView Function 75
9.18 MAID StopLiveView Function 76
9.19 MAID GetLiveViewStatus Function 76
9.20 MAID StartShooting Function 77
9.21 MAID CancelShooting Function 78
9.22 MAID GetShootingStatus Function 79
9.23 MAID GetRemainingShootCount Function 79
9.24 MAID GetCapability Function 79
9.25 MAID SetCapability Function 80
9.26 MAID EnumCapabilities Function 81
9.27 MAID EnumDevices Function 82
9.28 MAID GetLoggingLevel Function 82
9.29 MAID SetLoggingLevel Function 83
9.30 MAID StartMovieRecording Function 83
9.31 MAID StopMovieRecording Function 84
9.32 MAID StartOperation Function 84
9.33 MAID MarkAsynCallSequence Function 85
9.34 MAID AbortAsynCallOrSequence Function 85
9.35 MAID SetImageVideoSavePath Function 85
9.36 MAID IsDeviceFullSupport Function 86

10 History 87

10.1 New 87

1 Authors

2 Usage

To simplify the examples shown here, all calls to the module will be made synchronously. Asynchronous operation is explained below.

2.1 Module Loading

The process of loading a module is system dependent.

About the code how to load module, please refer the function, Load_Module(), Function.cpp, in Sample program.

2.2 Module Initialization

The first step is to have an object of NkMAIDCSCallback. Below is the structure of NkMAIDCSCallback.

```
	typedef struct tagNkMAIDCSCallback
	{
		LPNKFUNC	pUIReqProc; //Type - LPMAIDUIRequestProc
		LPNKFUNC	pfnEventProc; //Type - LPMAIDEventProc
		LPNKFUNC	pProgressProc; //Type - LPMAIDProgressProc
		LPNKFUNC	pDataProc;  //Type - LPMAIDDataProc
		LPNKFUNC	pLiveViewDataProc; //Type - LPLiveViewDataProc
		NKREF		refProc;
	} NkMAIDCSCallback, FAR* LPNkMAIDCSCallback;
```

The client will initialize the structure by setting the pfnEventProc member (Line 2), pUIReqProc member (Line 3), pProgressProc member (Line 4), pDataProc member (Line 5), pLiveViewDataProc member (Line 6), refProc member (Line 7).

It will then call the InitializeSDK (Line 9) with a AllocateMemory(), FreeMemory() function pointers, pointer to the NkMAIDCSCallback , pointer to the NkMAIDEnumDevices and NULL as a last parameter.

AllocateMemory() is the Function pointer of memory allocation API.

FreeMemory() is the Function pointer of memory free API allocated by AllocateMemory().

```
1	NkMAIDCSCallback callback;
2	callback.pfnEventProc = (LPNKFUNC)SrcEventProc;
3	callback.pUIReqProc = (LPNKFUNC)UIRequestProc;
4	callback.pProgressProc = (LPNKFUNC)ProgressProc;
5	callback.pDataProc = (LPNKFUNC)DataProc;
6	callback.pLiveViewDataProc = (LPNKFUNC)LVDataProc;
7	callback.refProc = NULL;
8
9	lerror=g_pMAIDInitializeSDK((LPMAIDAllocateMemory)AllocateMemory, (LPMAIDFreeMemory)FreeMemory,
&callback, &pDeviceList, NULL);
10	if (lerror != kNkMAIDResult_NoError)
11	{
12		puts("Unable to Initialize SDK.\n");
13		return -1;
14	}
```

2.3 Device Connection

After initialization, we need to connect with a device. To connect device client will call ConnectDevice() API (Line 1) with DeviceID and pointer to structure of NkMAIDEnumCapInfo as parameter. If connection is successful then ConnectDevice() API, allocate the capability information in pEnumCapInfo.

```
1       lerror=g_pMAIDConnectDevice(DeviceID, pEnumCapInfo);
2       if (lerror != kNkMAIDResult_NoError)
3       {
4               printf("Unable to Connect with Device.\n");
5               return -1;
6       }
```

2.4 Capability Enumeration

The process of reading Enum capabilities is similar to reading capability information. The client must check that the capability is Enum type and that it supports the kNkMAIDCommand_CapGetArray and kNkMAIDCommand_CapGet commands (Line 8 and 9). Client call GetCapability() (Line 15) with CapId, request type, pointer to capValue and datatype as a parameter. Once the call is successful then depending on array type client can call SetCapability() (Line 75, 84, 91, 104) with cap id, new value and type as parameter.

```
1 BOOL SetEnumCapability(ULONG ulCapID,LPVOID pCapValue,eNkMAIDDataType eDataType)
2 {
3       int new_val;
4       char buf[256];
5       NKERROR lerror = kNkMAIDResult_NoError;
6       bool set_cap_applicable = false;
7       bool GetSettingSupportedValueArray = false;
8       GetSettingSupportedValueArray=CheckCapability(ulCapID, NkMAIDCapOperation_GetArray);
9       set_cap_applicable = CheckCapability(ulCapID, kNkMAIDCapOperation_Get);
10      if (!set_cap_applicable)
11              return false;
12      NkMAIDEnum      stEnum = { 0 };
13      if (GetSettingSupportedValueArray)
14      {
15              lerror=g_pMAIDGetCapability(ulCapID, kNkSDKGetSettingSupportedValueArray, pCapValue,
eDataType);
16              if (lerror != kNkMAIDResult_NoError || pCapValue == NULL)
17              {
18                      if (pCapValue) freeByDataType(pCapValue, eDataType);
19                      printf("Failed to get capability value\n");
20                      return false;
21              }
22              else
23              {
24                      if (eDataType != kNkMAIDDataType_EnumPtr)
25                      {
26                              printf("Invalid data type returned\n");
27                              freeByDataType(pCapValue, eDataType);
28                              return false;
29                      }
30                      else
31                      {
32                              stEnum = *((NkMAIDEnum*)pCapValue);
33                              if (printEnumCapArrayValues(ulCapID, stEnum) == false)
34                              {
35                                      freeByDataType(pCapValue, eDataType);
36                                      return FALSE;
37                              }
38                              freeByDataType(pCapValue, eDataType);
39                      }
40              }
41      }
42      lerror=g_pMAIDGetCapability(ulCapID,kNkSDKGetSettingValue,pCapValue, eDataType);
43      if (lerror != kNkMAIDResult_NoError || pCapValue == NULL)
```

```
44      {
45              if (pCapValue) freeByDataType(pCapValue, eDataType);
46              printf("Failed to get capability value\n");
47              return false;
48      }
49      else
50      {
51              if (eDataType != kNkMAIDDataType_EnumPtr)
52              {
53                      printf("Invalid data type returned\n");
54                      freeByDataType(pCapValue, eDataType);
55                      return false;
56              }
57              else
58              {
59                      stEnum = *((NkMAIDEnum*)pCapValue);
60                      printf("Cap Value = %d\n", stEnum.ulValue);
61              }
62              freeByDataType(pCapValue, eDataType);
63      }
64      set_cap_applicable = CheckCapability(ulCapID, kNkMAIDCapOperation_Set);
65      if (!set_cap_applicable)
66              return false;
67      switch (stEnum.ulType)
68      {
69      case kNkMAIDArrayType_Unsigned:
70      {
71              printf("Input new value\n>");
72              scanf("%s", buf);
73              stEnum.ulValue = atoi(buf);
74              stEnum.pData = NULL;
75              lerror=g_pMAIDSetCapability(ulCapID,&stEnum,kNkMAIDDataType_EnumPtr);
76      }
77      break;
78      case kNkMAIDArrayType_PackedString:
79      {
80              printf("Input new value\n>");
81              scanf("%s", buf);
82              stEnum.ulValue = atoi(buf);
83              stEnum.pData = NULL;
84              lerror = g_pMAIDSetCapability(ulCapID, &stEnum, kNkMAIDDataType_EnumPtr);
85      }
86      break;
87      case kNkMAIDArrayType_String:
88      {
89              printf("Input new value\n>");
90              scanf("%s", buf);
91              lerror = g_pMAIDSetCapability(ulCapID, buf, kNkMAIDDataType_StringPtr);
92      }
93      break;
94      case kNkMAIDArrayType_Boolean:
95      {
96              printf("Input new value\n>");
97              printf("Input '1' to set True \n");
98              printf("Input '2' to set False \n");
99              scanf("%s", buf);
100             new_val = atoi(buf);
101             if ((new_val == 1) || (new_val == 2))
102             {
103                     unsigned char bFlag = (new_val == 1) ? true : false;
104                     lerror=g_pMAIDSetCapability(ulCapID,&bFlag,kNkMAIDDataType_BooleanPtr);
105             }
106         }
107         break;
108     }
109     free(stEnum.pData);
110     return TRUE;
111 }
```

2.5 Shooting Still Images (Single Shooting)

Client can Start Shooting only after connecting the device. To start shooting client will have MAIDShootingStructure object. Client will assign MAIDShootingStructure object ShootingType to Single (Line 7). Then client will set ImageSavePath to set the transferred image storage location(Line 6). Then client will call StartShooting() (Line 8) with first Parameter to stShootParam and other two parameters to NULL for Synchronous shooting.

```
1.  MAIDShootingStructure stShootParam;
2.  eNkMAIDShootingStatus ShootingStatus = kNkSDKShootingFinishedOrNotStarted;
3.  //Single Shooting
4.  printf("Single Shooting\n");
5.  memset(&stShootParam, 0, sizeof(MAIDShootingStructure));
6.  wcscpy(stShootParam.ImageSavePath, L"");//give a directory path
7.  stShootParam.ShootingType = kNkSDKShootingTypeSingle;
8.  lerror = g_pMAIDStartShooting(stShootParam, NULL, NULL);
9.  if(lerror == kNkMAIDResult_Waiting_2ndRelease)
10. printf("Waiting for 2nd Release.\n");
11. else if (lerror != kNkMAIDResult_NoError)
12. printf("Unable to start single shooting.\n");
```

2.6 Data Transfer

Image Saving is responsibility of SDK. SDK will save the images on specified path. Once image is saved, SDK will send the event kNkMAIDEvent_ImageSaved with image name as data.

2.7 Disconnect Device

Once user performed all the operation, before switching device or removing device client must call DisConnectDevice() . DisConnectDevice() will free all the memory allocated for the device and send command to device for Disconnection.

```
1   HRESULT hRes = g_pMAIDDisconnectDevice();
```

2.8 Module Termination

Whenever client want to close application, client must call FreeSDK() to free all the memory and object created by SDK.

```
1   HRESULT hRes = g_pMAIDFreeSDK();
```

3 Enumerations

3.1 Result Codes

```
enum eNkMAIDResult
{
	// these values are errors
	kNkMAIDResult_NotSupported = -127,
	kNkMAIDResult_UnexpectedDataType,
	kNkMAIDResult_ValueOutOfBounds,
	kNkMAIDResult_BufferSize,
	kNkMAIDResult_Aborted,
	kNkMAIDResult_NoMedia,
	kNkMAIDResult_NoEventProc,
	kNkMAIDResult_NoDataProc,
	kNkMAIDResult_ZombieObject,
	kNkMAIDResult_OutOfMemory,
	kNkMAIDResult_UnexpectedError,
	kNkMAIDResult_HardwareError,
	kNkMAIDResult_MissingComponent,
	//New Error code
	kNkMAIDAPIResult_DeviceNotAvailable,
	kNkMAIDAPIResult_OpenSessionFailed,
	kNkMAIDAPIResult_LiveViewAlreadyStarted,
	kNkMAIDAPIResult_LiveViewAlreadyStopped,
	kNkMAIDAPIResult_InvalidDeviceID,
	kNkMAIDAPIResult_StartLiveViewFailed,
	kNkMAIDAPIResult_DeviceFailed,
	kNkMAIDAPIResult_CapabilityNotSupported,
	kNkMAIDAPIResult_OperationNotSupported,
	kNkMAIDAPIResult_InvalidCapID,
	kNkMAIDAPIResult_OutOfRangeValue,
	kNkMAIDAPIResult_InvalidShootingType,
	kNkMAIDAPIResult_ShootingCancelFailed,
	kNkMAIDAPIResult_InvalidShootCount,
	kNkMAIDAPIResult_InvalidShootingMode,
	kNkMAIDAPIResult_InvalidFileSavePath,
	kNkMAIDAPIResult_MovieStartFailed,
	kNkMAIDAPIResult_FileSaveFailed,
	kNkMAIDAPIResult_InvalidInterval, //Change
	kNkMAIDAPIResult_InvalidStartTime, //Change
	kNkMAIDAPIResult_InvalidBULBExposureTime, //Change
	kNkMAIDAPIResult_InvalidArguments, //Change
	kNkMAIDAPIResult_InvalidState, //Change
	kNkMAIDAPIResult_ContinousShootIntrupted,
	//New  Error code
	kNkMAIDResult_NoError = 0,

	// these values are warnings
	kNkMAIDResult_Pending,
	kNkMAIDResult_OrphanedChildren,

	kNkMAIDResult_VendorBase = +127
};
```

The module will deliver one of these values to the **nResult** parameter of the client's completion callback function and return the same value from the entry point. Errors will have negative values.

3.2 Data Object Types

```
enum eNkMAIDDataObjType
{
	kNkMAIDDataObjType_Image		= 0x00000001,
	kNkMAIDDataObjType_Sound		= 0x00000002,
	kNkMAIDDataObjType_Video		= 0x00000004,
	kNkMAIDDataObjType_Thumbnail	= 0x00000008,
	kNkMAIDDataObjType_File		= 0x00000010
};
```

The module will use one or more of these values to indicate what types of data a module or source can produce and what types of data are available for a specific item. See the description of *kNkMAIDCapability_DataTypes* for more information.

3.3 Data Types

```
enum eNkMAIDDataType
{
	kNkMAIDDataType_Null = 0,
	kNkMAIDDataType_Boolean,		// passed by value, set only
	kNkMAIDDataType_Integer,		// signed 32 bit int, passed by value, set only
	kNkMAIDDataType_Unsigned,		// unsigned 32 bit int, passed by value, set only
	kNkMAIDDataType_BooleanPtr,	// pointer to single byte boolean value(s)
	kNkMAIDDataType_IntegerPtr,	// pointer to signed 4 byte value(s)
	kNkMAIDDataType_UnsignedPtr,	// pointer to unsigned 4 byte value(s)
	kNkMAIDDataType_FloatPtr,		// pointer to DOUB_P value(s)
	kNkMAIDDataType_PointPtr,		// pointer to NkMAIDPoint structure(s)
	kNkMAIDDataType_SizePtr,		// pointer to NkMAIDSize structure(s)
	kNkMAIDDataType_RectPtr,		// pointer to NkMAIDRect structure(s)
	kNkMAIDDataType_StringPtr,	// pointer to NkMAIDString structure(s)
	kNkMAIDDataType_DateTimePtr,	// pointer to NkMAIDDateTime structure(s)
	kNkMAIDDataType_CallbackPtr,	// pointer to NkMAIDCallback structure(s)
	kNkMAIDDataType_RangePtr,		// pointer to NkMAIDRange structure(s)
	kNkMAIDDataType_ArrayPtr,		// pointer to NkMAIDArray structure(s)
	kNkMAIDDataType_EnumPtr,		// pointer to NkMAIDEnum structure(s)
	kNkMAIDDataType_ObjectPtr,	// pointer to NkMAIDObject structure(s)
	kNkMAIDDataType_CapInfoPtr,	// pointer to NkMAIDCapInfo structure(s)
	kNkMAIDDataType_GenericPtr,	// pointer to generic data
	kNkMAIDDataType_StringStepsPtr
};
```

The client will pass one of these values to the **ulDataType** parameter of the entry point to indicate how the **data** parameter will be interpreted by the module.

3.4 Array Types

```
enum eNkMAIDArrayType
{
	kNkMAIDArrayType_Boolean = 0,	// 1 byte per element
	kNkMAIDArrayType_Integer,	// signed value that is 1, 2 or 4 bytes per element
	kNkMAIDArrayType_Unsigned,	// unsigned value that is 1, 2 or 4 bytes per element
	kNkMAIDArrayType_Float,	// DOUB_P elements
	kNkMAIDArrayType_Point,	// NkMAIDPoint structures
	kNkMAIDArrayType_Size,	// NkMAIDSize structures
	kNkMAIDArrauType_Rect,	// NkMAIDRect structures
	kNkMAIDArrayType_PackedString,	// packed array of strings
	kNkMAIDArrayType_String,	// NkMAIDString structures
	kNkMAIDArrayType_DateTime	// NkMAIDDateTime structures
};
```

The module will set one of these values in the **ulType** member of the NkMAIDArray structure to indicate how the data of the array should be interpreted. See the description of the NkMAIDArray structure for more information.

3.5 Capability Types

```
enum eNkMAIDCapType
{
	kNkMAIDCapType_Process = 0,	// a process that can be started
	kNkMAIDCapType_Boolean,	// single byte boolean value
	kNkMAIDCapType_Integer,	// signed 4 byte value
	kNkMAIDCapType_Unsigned,	// unsigned 4 byte value
	kNkMAIDCapType_Float,	// DOUB_P value
	kNkMAIDCapType_Point,	// NkMAIDPoint structure
	kNkMAIDCapType_Size,	// NkMAIDSize structure
	kNkMAIDCapType_Rect,	// NkMAIDRect structure
	kNkMAIDCapType_String,	// NkMAIDString structure
	kNkMAIDCapType_DateTime,	// NkMAIDDateTime structure
	kNkMAIDCapType_Callback,	// NkMAIDCallback structure
	kNkMAIDCapType_Array,	// NkMAIDArray structure
	kNkMAIDCapType_Enum,	// NkMAIDEnum structure
	kNkMAIDCapType_Range,	// NkMAIDRange structure
	kNkMAIDCapType_Generic,	// generic data
	kNkMAIDCapType_BoolDefault,	// Reserved
	kNkMAIDCapType_StringSteps
};
```

The module will set one of these values in the **ulType** member of the NkMAIDCapInfo structure to indicate what type of information is represented. See the Capabilities chapter and the description of the NkMAIDCapInfo structure for more information.

3.6 Capability Operations

```
enum eNkMAIDCapOperations
{
	kNkMAIDCapOperation_Start	= 0x0001,
	kNkMAIDCapOperation_Get	= 0x0002,
	kNkMAIDCapOperation_Set	= 0x0004,
	kNkMAIDCapOperation_GetArray,	= 0x0008,
	kNkMAIDCapOperation_GetDefault	= 0x0010
};
```

The module will set one of more of these values in the **ulOperations** member of the NkMAIDCapInfo structure to indicate what operations are permitted on a particular capability. See the Capabilities chapter and the description of the NkMAIDCapInfo structure for more information.

3.7 Capability Visibility

```
enum eNkMAIDCapVisibility
{
	kNkMAIDCapVisibility_Hidden	= 0x0001,
	kNkMAIDCapVisibility_Advanced	= 0x0002,
	kNkMAIDCapVisibility_Vendor	= 0x0004,
	kNkMAIDCapVisibility_Group	= 0x0008,
	kNkMAIDCapVisibility_GroupMember = 0x0010,
	kNkMAIDCapVisibility_Invalid	= 0x0020
};
```

The module will set one or more of these values in the **ulVisibility** member of the NkMAIDCapInfo structure to indicate what level of visibility a particular capability has. See the Capabilities chapter and the description of the NkMAIDCapInfo structure for more information.

3.8 Object Types

```
enum eNkMAIDObjectType
{
	kNkMAIDObjectType_Module = 1,
	kNkMAIDObjectType_Source,
	kNkMAIDObjectType_Item,
	kNkMAIDObjectType_DataObj
};
```

The module will set one of these values in the **ulType** member of the NkMAIDObject structure to indicate what type of object is represented.

3.9 Events

```
enum eNkMAIDEvent
{
	kNkMAIDEvent_AddChild,
	kNkMAIDEvent_RemoveChild,
	kNkMAIDEvent_WarmingUp,
	kNkMAIDEvent_WarmedUp,
	kNkMAIDEvent_CapChange,
	kNkMAIDEvent_OrphanedChildren,
	kNkMAIDEvent_CapChangeValueOnly
	kNkMAIDEvent_DeviceInfoChanged,  //Added New
	kNkMAIDEvent_ImageSaved,          //Added New
	kNkMAIDEvent_ImageSavedInCard, //Added New
	kNkMAIDEvent_StorageFull_ImageNotSaved, //Added New
	kNkMAIDEvent_AcquireFailed_ImageNotSaved, //Added New
	kNkMAIDEvent_ChangeSaveDestination,
	kNkMAIDEvent_AddImageInCard
};
```

The module will deliver one of these values to the **ulEvent** parameter of the client's event callback function to indicate what event has occurred.

3.10 User Interface Request Types

```
enum eNkMAIDUIRequestType
{
	kNkMAIDUIRequestType_Ok,
	kNkMAIDUIRequestType_OkCancel,
	kNkMAIDUIRequestType_YesNo,
	kNkMAIDUIRequestType_YesNoCancel,
};
```

When the module calls the client's user interface callback function, the **ulType** member of the NkMAIDUIRequestInfo structure will be set to one of these values. The user will be presented with the choices specified by the value.

3.11 User Interface Results

```
enum eNkMAIDUIRequestResult
{
	kNkMAIDUIRequestResult_None,
	kNkMAIDUIRequestResult_Ok,
	kNkMAIDUIRequestResult_Cancel,
	kNkMAIDUIRequestResult_Yes,
	kNkMAIDUIRequestResult_No
};
```

When the module calls the client's user interface callback function, the **ulDefault** member of the NkMAIDUIRequestInfo structure will be set to one of these values. The value will indicate which button should be highlighted by default. The client's user interface callback function will return one of these values depending on which button the user presses. The *kNkMAIDEventResult_None* value can only be returned if the **fSync** member of the NkMAIDUIRequestInfo structure is FALSE.

3.12 Filters

```
enum eNkMAIDFilter
{
	kNkMAIDFilter_White,
	kNkMAIDFilter_Infrared,
	kNkMAIDFilter_Red,
	kNkMAIDFilter_Green,
	kNkMAIDFilter_Blue,
	kNkMAIDFilter_Ultraviolet
};
```

The module will use one or more of these values in the kNkMAIDCapability_Filter capability. See the Capabilities chapter for more information.

3.13 Commands

```
enum eNkMAIDCommand
{
	kNkMAIDCommand_Async,			// process asynchronous operations
	kNkMAIDCommand_Open,			// opens a child object
	kNkMAIDCommand_Close,			// closes a previously opened object
	kNkMAIDCommand_GetCapCount,		// get number of capabilities of an object
	kNkMAIDCommand_GetCapInfo,		// get capabilities of an object
	kNkMAIDCommand_CapStart,		// starts capability
	kNkMAIDCommand_CapSet,			// set value of capability
	kNkMAIDCommand_CapGet,			// get value of capability
	kNkMAIDCommand_CapGetDefault,	// get default value of capability
	kNkMAIDCommand_CapGetArray		// get data for array capability
	kNkMAIDCommand_Mark,			// insert mark in the command queue
	kNkMAIDCommand_AbortToMark,		// abort asynchronous commands to mark
	kNkMAIDCommand_Abort,			// abort current asynchronous command
	kNkMAIDCommand_EnumChildren,	// requests 'add' events for all child IDs
	kNkMAIDCommand_GetParent,		// gets previously opened parent for object
	kNkMAIDCommand_ResetToDefault	// resets all capabilities to their default value
};
```

The client will pass one of these values to the **ulCommand** parameter of the MAID entry point to indicate what operation the module should perform. These commands are explained in detail in the Commands chapter.

3.14 Capabilities

```
enum eNkMAIDCapability
{
	kNkMAIDCapability_AsyncRate = 1,  // milliseconds between idle async calls
	kNkMAIDCapability_ProgressProc,   // callback during lengthy operation
	kNkMAIDCapability_EventProc,      // callback when event occurs
	kNkMAIDCapability_DataProc,       // callback to deliver data
	kNkMAIDCapability_UIRequestProc,  // callback to show user interface

	kNkMAIDCapability_IsAlive,        // FALSE if object is removed or parent closed
	kNkMAIDCapability_Children,       // IDs of children objects
	kNkMAIDCapability_State,          // current state of the object
	kNkMAIDCapability_Name,           // a string representing the name of the object
	kNkMAIDCapability_Description,    // a string describing the object
	kNkMAIDCapability_Interface,      // a string describing the interface to a device
	kNkMAIDCapability_DataTypes,      // what data types are supported or available
	kNkMAIDCapability_DateTime,       // date associated with an object
	kNkMAIDCapability_StoredBytes,    // read only size of object as stored on device

	kNkMAIDCapability_Eject,          // ejects media from a device
	kNkMAIDCapability_Feed,           // feeds media into a device
	kNkMAIDCapability_Capture,        // captures new item from the source
	kNkMAIDCapability_MediaPresent,   // TRUE if item has physical media to acquire

	kNkMAIDCapability_Mode,           // mode of this object
	kNkMAIDCapability_Acquire,        // begins the acquisition of the object
	kNkMAIDCapability_ForceScan,      // If FALSE, unneccesary scans can be eliminated

	kNkMAIDCapability_Start,          // start offset (in seconds) of the object
	kNkMAIDCapability_Length,         // length (in seconds) of the object
	kNkMAIDCapability_SampleRate,     // sampling rate (in samples/sec) of the object
	kNkMAIDCapability_Stereo,         // mono or stereo
	kNkMAIDCapability_Samples,        // given current state, read only number of samples

	kNkMAIDCapability_Filter,         // selects the filter for the light source
	kNkMAIDCapability_Prescan,        // sets the focus automatically
	kNkMAIDCapability_ForcePrescan,   // If FALSE, unneccesary prescans can be eliminated
	kNkMAIDCapability_ForceAutoFocus, // If FALSE, unneccesary autofocus can be eliminated
	kNkMAIDCapability_AutoFocusPt,    // sets the position to focus upon
	kNkMAIDCapability_Focus,          // sets the focus
	kNkMAIDCapability_Coords,         // rectangle of object in device units
	kNkMAIDCapability_Resolution,     // resolution of object (in pixels/inch)
	kNkMAIDCapability_Preview,        // preview or final acquire
	kNkMAIDCapability_Negative,       // negative or positive original
	kNkMAIDCapability_ColorSpace,     // color space
	kNkMAIDCapability_Bits,           // bits per color
	kNkMAIDCapability_Planar,         // interleaved or planar data transfer
	kNkMAIDCapability_Lut,            // LUT(s) for object
	kNkMAIDCapability_Transparency,   // light path of the original
	kNkMAIDCapability_Threshold,      // threshold level for lineart images
	kNkMAIDCapability_Pixels,         // given current state, read only size of image
	kNkMAIDCapability_NegativeDefault,// Default value for Negative capbility
	kNkMAIDCapability_Firmware,       // Firmware version

	kNkMAIDCapability_CommunicationLevel1,  // Device communication method (level 1)
	kNkMAIDCapability_CommunicationLevel2,  // Device communication method (level 2)
	kNkMAIDCapability_BatteryLevel,         // Device battery level
	kNkMAIDCapability_FreeBytes,            // Free bytes in device storage
	kNkMAIDCapability_FreeItems,            // Free items in device storage
	kNkMAIDCapability_Remove,               // Remove an object from device storage
	kNkMAIDCapability_FlashMode,            // Device flash (speedlight) mode
	kNkMAIDCapability_ModuleType,           // Module type
	kNkMAIDCapability_AcquireStreamStart,   // Start a stream acquire
	kNkMAIDCapability_AcquireStreamStop,    // Stop a stream acquire
```

```
        kNkMAIDCapability_AcceptDiskAcquisition,// Sets parameters for disk acquisition
        kNkMAIDCapability_Version,              // MAID version
        kNkMAIDCapability_FilmFormat,           // Film format (35mm, 6*6 etc)
        kNkMAIDCapability_TotalBytes,           // Number of bytes in device storage
        kNkMAIDCapability_AutoFocus,            // Auto Focus
        kNkMAIDCapability_VendorBase = 0x8000   // vendor supplied capabilities start here
};
```

The module will use these values in the **ulID** member of NkMAIDCapInfo structures to indicate what capabilities it provides to the client. The client will pass one of these values to the **ulParam** parameter of the MAID entry point to indicate what capability to perform the command upon.

The module writer can define capabilities unique to their device. The client will allow the user to interact with those capabilities in a generic manner.

These capabilities are explained in detail in the Capabilities chapter.

3.15 Color Spaces

```
enum eNkMAIDColorSpace
{
        kNkMAIDColorSpace_LineArt,
        kNkMAIDColorSpace_Grey,
        kNkMAIDColorSpace_RGB,
        kNkMAIDColorSpace_sRGB,
        kNkMAIDColorSpace_CMYK,
        kNkMAIDColorSpace_Lab,
        kNkMAIDColorSpace_LCH,
        kNkMAIDColorSpace_AppleRGB,
        kNkMAIDColorSpace_ColorMatchRGB,
        kNkMAIDColorSpace_NTSCRGB,
        kNkMAIDColorSpace_BruceRGB,
        kNkMAIDColorSpace_AdobeRGB,
        kNkMAIDColorSpace_CIERGB,
        kNkMAIDColorSpace_AdobeWideRGB,
        kNkMAIDColorSpace_AppleRGB_Compensated

        kNkMAIDColorSpace_VendorBase = 0x8000   // vendor supplied colorspaces start here
};
```

The client will use these values in the kNkMAIDCapability_ColorSpace capability. See the Capabilities chapter for more information.

3.16 Boolean Defaults

```
enum eNkMAIDBooleanDefault
{
        kNkMAIDBooleanDefault_None,
        kNkMAIDBooleanDefault_True,
        kNkMAIDBooleanDefault_False
};
```

The client will use these values with the kNkMAIDCapability_NegativeDefault capability. See the Capabilities chapter for more information.

3.17 Module Types

```
enum eNkMAIDModuleTypes
{
	kNkMAIDModuleType_Scanner	= 0x0001,
	kNkMAIDModuleType_Camera	= 0x0002
};
```

The module will return one of more of these values in the *kNkCapability_ModuleType* capability. This will help the client determine if this module should be used, or perhaps which user interface to display.

3.18 File Data Types

```
enum eNkMAIDFileDataTypes
{
	kNkMAIDFileDataType_NotSpecified,
	kNkMAIDFileDataType_JPEG,
	kNkMAIDFileDataType_TIFF,
	kNkMAIDFileDataType_FlashPix,
	kNkMAIDFileDataType_NIF,
	kNkMAIDFileDataType_QuickTime,
	kNkMAIDFileDataType_HEIF,
	kNkMAIDFileDataType_UserType = 0x100
};
```

The module will use these values in the NkMAIDFileInfo structure when sending file data to the client in response to the kNkMAIDCapability_Acquire capability.

3.19 Flash Modes

```
enum eNkMAIDFlashMode
{
	kNkMAIDFlashMode_FrontCurtain,
	kNkMAIDFlashMode_RearCurtain,
	kNkMAIDFlashMode_SlowSync,
	kNkMAIDFlashMode_RedEyeReduction,
	kNkMAIDFlashMode_SlowSyncRedEyeReduction,
	kNkMAIDFlashMode_SlowSyncRearCurtain
};
```

The client will use these values in the kNkMAIDCapability_FlashMode capability. See the Capabilities chapter for more information.

3.20 Fmm Manual Setting

```
enum eNkMAIDFmmManualSetting
{
	kNkMAIDFmmManualSetting_NoSettting = 0,
	kNkMAIDFmmManualSetting_6mm,
	kNkMAIDFmmManualSetting_8mm,
	kNkMAIDFmmManualSetting_13mm,
	kNkMAIDFmmManualSetting_15mm,
	kNkMAIDFmmManualSetting_16mm,
	kNkMAIDFmmManualSetting_18mm,
	kNkMAIDFmmManualSetting_20mm,
	kNkMAIDFmmManualSetting_24mm,
	kNkMAIDFmmManualSetting_25mm,
	kNkMAIDFmmManualSetting_28mm,
```

```
	kNkMAIDFmmManualSetting_35mm,
	kNkMAIDFmmManualSetting_43mm,
	kNkMAIDFmmManualSetting_45mm,
	kNkMAIDFmmManualSetting_50mm,
	kNkMAIDFmmManualSetting_55mm,
	kNkMAIDFmmManualSetting_58mm,
	kNkMAIDFmmManualSetting_70mm,
	kNkMAIDFmmManualSetting_80mm,
	kNkMAIDFmmManualSetting_85mm,
	kNkMAIDFmmManualSetting_86mm,
	kNkMAIDFmmManualSetting_100mm,
	kNkMAIDFmmManualSetting_105mm,
	kNkMAIDFmmManualSetting_135mm,
	kNkMAIDFmmManualSetting_180mm,
	kNkMAIDFmmManualSetting_200mm,
	kNkMAIDFmmManualSetting_300mm,
	kNkMAIDFmmManualSetting_360mm,
	kNkMAIDFmmManualSetting_400mm,
	kNkMAIDFmmManualSetting_500mm,
	kNkMAIDFmmManualSetting_600mm,
	kNkMAIDFmmManualSetting_800mm,
	kNkMAIDFmmManualSetting_1000mm,
	kNkMAIDFmmManualSetting_1200mm,
	kNkMAIDFmmManualSetting_1400mm,
	kNkMAIDFmmManualSetting_1600mm,
	kNkMAIDFmmManualSetting_2000mm,
	kNkMAIDFmmManualSetting_2400mm,
	kNkMAIDFmmManualSetting_2800mm,
	kNkMAIDFmmManualSetting_3200mm,
	kNkMAIDFmmManualSetting_4000mm
};
```

3.21 F0 Manual Setting

```
enum eNkMAIDF0ManualSetting
{
	kNkMAIDF0ManualSetting_NoSettting = 0,
	kNkMAIDF0ManualSetting_F1_2,
	kNkMAIDF0ManualSetting_F1_4,
	kNkMAIDF0ManualSetting_F1_8,
	kNkMAIDF0ManualSetting_F2_0,
	kNkMAIDF0ManualSetting_F2_5,
	kNkMAIDF0ManualSetting_F2_8,
	kNkMAIDF0ManualSetting_F3_3,
	kNkMAIDF0ManualSetting_F3_5,
	kNkMAIDF0ManualSetting_F4_0,
	kNkMAIDF0ManualSetting_F4_5,
	kNkMAIDF0ManualSetting_F5_0,
	kNkMAIDF0ManualSetting_F5_6,
	kNkMAIDF0ManualSetting_F6_3,
	kNkMAIDF0ManualSetting_F7_1,
	kNkMAIDF0ManualSetting_F8_0,
	kNkMAIDF0ManualSetting_F9_5,
	kNkMAIDF0ManualSetting_F11,
	kNkMAIDF0ManualSetting_F13,
	kNkMAIDF0ManualSetting_F15,
	kNkMAIDF0ManualSetting_F16,
	kNkMAIDF0ManualSetting_F19,
	kNkMAIDF0ManualSetting_F22
};
```

3.22 LV Status

```
enum eNkMAIDLVStatus
{
	eNkMAIDLVStatus_OFF = 0,
	eNkMAIDLVStatus_ON,
	eNkMAIDLVStatus_ON_Menu,
	eNkMAIDLVStatus_ON_RemoteLV,
	eNkMAIDLVStatus_ON_CameraLV
};
```

4 Structures and Types

4.1 Word Value

```
typdef unsigned short WORD;
```

This definition may be implementation dependent with the only requirement that it must be appropriate for 16 bit unsigned integers.

4.2 Unsigned Long Value

```
typdef unsigned long ULONG;
```

This definition may be implementation dependent with the only requirement that it must be appropriate for 32 bit unsigned integers.

4.3 Parameter Value

```
typdef ULONG NKPARAM;
```

This definition may be implementation dependent with the only requirement that it must be appropriate for pointers to objects and 32 bit integers.

4.4 Pointer Value

```
typdef void FAR *LPVOID;
```

This definition may be implementation dependent with the only requirement that it must be appropriate for pointers to objects.

4.5 Reference Value

```
typedef LPVOID NKREF;
```

This definition may be implementation dependent with the only requirement that it must be appropriate for pointers to objects. This type will be used in structures where the client wishes to associate the structure with another structure or object. This is also used by callback functions.

4.6 MAID Entry Point Function Pointer

```
typedef NKERROR (FAR *LPMAIDEntryPointProc)( LPNkMAIDObject, ULONG, ULONG, ULONG, NKPARAM,
LPNKFUNC, NKREF );
```

4.7 MAID Completion Function Pointer

```
typedef void (FAR *LPMAIDCompletionProc)( LPNkMAIDObject, ULONG, ULONG, ULONG, NKPARAM, NKREF,
NKERROR );
```

4.8 MAID Data Delivery Function Pointer

```
typedef NKERROR (FAR *LPMAIDDataProc)( NKREF, LPVOID, LPVOID );
```

4.9 MAID Event Notification Function Pointer

```
typedef void (FAR *LPMAIDEventProc)( NKREF, ULONG, NKPARAM );
```

4.10 MAID Progress Notification Function Pointer

```
typedef void (FAR *LPMAIDProgressProc)( ULONG, ULONG, NKREF, ULONG, ULONG );
```

4.11 MAID User Interface Request Function Pointer

```
typedef ULONG (FAR *LPMAIDUIRequestProc)( NKREF, LPNkMAIDUIRequestInfo );
```

4.12 Callback Definition Structure

```
typedef struct tagNkMAIDCallback
{
       LPNKFUNC     pProc;
       NKREF        refProc;
} NkMAIDCallback, FAR* LPNkMAIDCallback;
```

This structure is used to describe a callback function. The **pProc** member points to the function and the **refProc** member is used by the callback for it's own purposes. Usually, **refProc** is used by the client to point to an object or structure.

4.13 Date/Time Structure

```
typedef struct tagNkMAIDDateTime
{
       UWORD nYear;         // ie 1997, 1998
       UWORD nMonth;        // 1-11 = Jan-Dec
       UWORD nDay;          // 1-31
       UWORD nHour;         // 0-23
       UWORD nMinute;       // 0-59
       UWORD nSecond;       // 0-59
       ULONG nSubsecond;    // Module dependent
} NkMAIDDateTime, FAR* LPNkMAIDDateTime;
```

Each individual module can decide how to interpret the **nSubsecond** member. For example, if several pictures were taken within one second, they could be assigned sequential numbers starting with zero. Alternatively, it could be implemented as milliseconds.

4.14 Point Structure

```
typedef struct tagNkMAIDPoint
{
	SLONG  x;
	SLONG  y;
} NkMAIDPoint, FAR* LPNkMAIDPoint;
```

4.15 Size Structure

```
typedef struct tagNkMAIDSize
{
	ULONG  w;
	ULONG  h;
} NkMAIDSize, FAR* LPNkMAIDSize;
```

4.16 Rectangle Structure

```
typedef struct tagNkMAIDRect
{
	SLONG  x;	// left coordinate
	SLONG  y;	// top coordinate
	ULONG  w;	// width
	ULONG  h;	// height
} NkMAIDRect, FAR* LPNkMAIDRect;
```

4.17 String Structure

```
typedef struct tagNkMAIDString
{
	SCHAR  str[256];	// allows a 255 character null terminated string
} NkMAIDString, FAR* LPNkMAIDString;
```

The string must be null terminated. Using this structure, the maximum length string that can be transferred is 255 characters long. Consider using the NkMAIDArray structure with the **ulType** member set to *kNkMAIDArrayType_PackedString* for longer strings.

4.18 Array Structure

```
typedef struct tagNkMAIDArray
{
	ULONG		ulType;			// one of eNkMAIDArrayType
	ULONG		ulElements;		// total number of elements
	ULONG		ulDimSize1;		// size of first dimension
	ULONG		ulDimSize2;		// size of second dimension, zero for 1 dim
	ULONG		ulDimSize3;		// size of third dimension, zero for 1 or 2 dim
	UWORD		wPhysicalBytes;	// bytes per element
	UWORD		wLogicalBits;		// must be <= wPhysicalBytes * 8
	LPVOID		pData;			// allocated by the client
} NkMAIDArray, FAR* LPNkMAIDArray;
```

The NkMAIDArray structure allows an array to be transferred through the MAID interface. The client will always allocate the memory. It is the responsibility of the receiver of the data to interpret the data properly. The size of **pData** in bytes should always be **ulElements** times **wPhysicalBytes**.

Two and three dimensional arrays may be transferred by setting **ulDimSize1**, **ulDimSize2** and **ulDimSize3**. For a two dimensional array of 20 rows of 10 elements each, **ulDimSize1** will be 10, **ulDimSize2** will be 20 and **ulElements** will be

200. If there are five of those arrays, **ulDimSize1** will be 10, **ulDimSize2** will be 20, **ulDimSize3** will be 5 and **ulElements** will be 1000.

When **ulType** is *kNkMAIDArrayType_Integer* or *kNkMAIDArrayType_Unsigned*, the sender may specify that although the integer values in **pData** are two bytes each (**wPhysicalBytes**), the receiver should interpret them as 10-bit values (**wLogicalBits**). The **wLogicalBits** member is ignored for other types.

When **ulType** is *kNkMAIDArrayType_PackedString*, **pData** will point to a packed list of null terminated strings, **ulElements** will be the total length in bytes of the data including the terminating null bytes, **ulDimSize1** will be the number of strings, **ulDimSize2** will be zero and **wPhysicalBytes** will be one.

4.19 Range Structure

```
typedef struct tagNkMAIDRange
{
       DOUB_P        lfValue;
       DOUB_P        lfDefault;
       ULONG         ulValueIndex;       // zero-based index
       ULONG         ulDefaultIndex;     // zero-based index
       DOUB_P        lfLower;
       DOUB_P        lfUpper;
       ULONG         ulSteps;     // zero for infinite range, otherwise must be >= 2
} NkMAIDRange, FAR* LPNkMAIDRange;
```

This structure is to implement capabilities with a numerical range of values (0-100, -5.0 to +5.0, etc.) The lower and upper limits will be in **lfLower** and **lfUpper** respectively.

If any value from **lfLower** to **lfUpper** is allowed, **lfValue** will be the current value, **lfDefault** will be the default value and **ulSteps** will be zero. In this case, **ulValueIndex** and **ulDefaultIndex** are not used.

If only discrete steps are allowed, **ulValueIndex** will be the index of the current step, **ulDefaultIndex** will be the index of the default step and **ulSteps** will be the number of equally spaced steps including the lower and upper limits. There must be at least two discrete steps. In this case, **lfValue** and **lfDefault** are not used.

There are two ways to set the value of a range. The client can send a pointer to a NkMAIDRange structure with a new value in **lfValue** or **ulValueIndex**. For ranges with discrete steps, the client can send an unsigned integer which the module will accept as the index of the value. In that case, zero is the lower limit and one less than **ulSteps** is the upper limit.

4.20 Capability Information Structure

```
typedef struct tagNkMAIDCapInfo
{
       ULONG  ulID;               // one of eNkMAIDCapability or vendor specified
       ULONG  ulType;             // one of eNkMAIDCababilityType
       ULONG  ulVisibility;       // eNkMAIDCapVisibility bits
       ULONG  ulOperations;       // eNkMAIDCapOperations bits
       SCHAR  szDescription[256]; // text describing the capability
} NkMAIDCapInfo, FAR* LPNkMAIDCapInfo;
```

Each of the module, source, item, image and sound objects have their own capabilities. Use the *kNkMAIDCommand_GetCapInfo* to retrieve an array of these structures. Each has a unique identifier value in **ulID**. The ID can be one of the eNkMAIDCapability values or a vendor specific value.

4.21 Object Structure

```
typedef struct tagNkMAIDObject
{
	ULONG		ulType;			// one of eNkMAIDObjectType
	ULONG		ulID;
	NKREF		refClient;
	NKREF		refModule;
} NkMAIDObject, FAR* LPNkMAIDObject;
```

This structure is able to represent the module, source, item, image and sound objects that pass between the MAID client and module.

To open an object, the client will allocate the memory required and fill **refClient** with whatever value it needs. The client then calls the module to open the object. During that call, the module will set **ulType** to the appropriate value, **ulID** to the ID of the object and **refModule** to whatever value it needs. While the object is open, the values of **refClient** and **refModule** will not change.

If a module, source, item, image or sound is opened a second time, the module will maintain a second set of capability values for the new object. Both NkMAIDObject structures will have the same **ulID** value. The module will be able to differentiate the first and second instances by the value of the **refModule** member. The client will be able to differentiate the two by the value of the **refClient** member.

4.22 User Interface Request Structure

```
typedef struct tagNkMAIDUIRequestInfo
{
	ULONG			ulType;		// one of eNkMAIDUIRequestType
	ULONG			ulDefault;	// default return value one of eNkMAIDUIRequestResult
	BOOL			fSync;		// TRUE if user must respond before returning
	SCHAR FAR *		lpPrompt;	// NULL terminated text to show to user
	SCHAR FAR *		lpDetail;	// NULL terminated text indicating more detail
	LPNkMAIDObject	pObject;	// module, source, item, or data object
	NKPARAM			data;		// Pointer to an NkMAIDArray structure
} NkMAIDUIRequestInfo, FAR* LPNkMAIDUIRequestInfo;
```

When the module wants to notify the user of some event or query a response from the user, it will call the client's user interface function with **pUIRequest** set to a pointer to this structure. The **ulType** member indicates what buttons to make available to the user. The **ulDefault** member indicates which button will initially be highlighted. If the **fSync** member is TRUE, the client must immediately display the dialog and wait for the user's response. If it is FALSE, the client can either return the *kNkMAIDEventResult_None* value and display the dialog at some later time or wait for the user's response.

The **lpPrompt** member will point to a null terminated string provided by the module. The pointer will be valid for the length of the user interface callback. The client must make a copy of the string in order to show the user interface asynchronously.

If more detailed information is available, the **lpDetail** member will point to a null terminated string provided by the module. The pointer will be valid for the length of the user interface callback. The client must make a copy of the string in order to show the user interface asynchronously. If more detailed information is not available, the **lpDetail** member should be set to NULL.

If the UI request is a simple message with no capabilities, the **pObject** and **data** members should be set to NULL. If capabilities are to be presented to the user, the **data** member will point to an NkMAIDArray structure allocated by the module. The array structure will contain one or more MAID capability identifiers (listed in eNkMAIDCapability). All of these capabilities must refer to the MAID object specified in **pObject**. The client will attempt to display these capabilities to the user. The array structure should be filled as follows:

```
1      ulType = kNkMAIDArrayType_Unsigned
2      ulElements = <number of capabilities to be displayed>
3      ulDimSize1 = <same as ulElements>
4      ulDimSize2 = 0
5      ulDimSize3 = 0
6      wPhysicalBytes = 4
7      wLogicalBits = 32
8      pData = <array of capability IDs allocated by the module>
```

The following table lists the likely client implementation for each type of UI Request:

Capability Type	Likely UI
Process	Button
Boolean	Check Box
Integer, Unsigned, Float, String	Edit Control
Point, Size, Rect	Custom UI
DateTime	Edit Controls or Custom UI
Callback	Undefined
Array	Radio Button Group
Range	Slider or Spin Control

4.23 Generic Data Delivery Structure

```
typedef struct tagNkMAIDDataInfo
{
       ULONG        ulType;             // one of eNkMAIDDataObjType
} NkMAIDDataInfo, FAR* LPNkMAIDDataInfo;
```

This structure is used in the NkMAIDImageInfo, NkMAIDSoundInfo, and NkMAIDFileInfo structures to indicate what type of data is being delivered to the client's data delivery callback function. If kNkMAIDDataObjType_File is combined with other value to make up the **ulType** member, it means that the accompanying data is formatted as a file and that the NkMAIDFileInfo structure should be used.

4.24 Image Data Delivery Structure

```
typedef struct tagNkMAIDImageInfo
{
       NkMAIDDataInfo      base;
       NkMAIDSize          szTotalPixels;      // total size of image to be transfered
       ULONG               ulColorSpace;       // One of eNkMAIDColorSpace
       NKMAIDRect          rData;              // Coords of data, (0,0) = top left
       ULONG               ulRowBytes;         // number of bytes per row of pixels
       UWORD               wBits[4];           // number of bits per plane per pixel
       UWORD               wPlane;             // see below for description
       BOOL                fRemoveObject;      // TRUE if the object should be removed
} NkMAIDImageInfo, FAR* LPNkMAIDImageInfo;
```

The module sets the pDataInfo parameter of the client's data delivery callback function to a pointer to this structure to describe the image data being delivered. The **ulColorSpace** and **wBits** members apply to the image as a whole. If only one plane of a color image is being transferred, **ulColorSpace** will be the color space of the entire image all of the elements of **wBits** will be set.

If the data is being delivered in kNkMAIDColorSpace_LineArt or kNkMAIDColorSpace_Grey, the **wPlane** parameter will be ignored. If the data is being delivered in one of the color formats and is being delivered one plane at a time, the **wPlane** parameter will indicate the plane being delivered. For RGB and sRGB: R=1, G=2, B=3. For CMYK: C=1, M=2, Y=3, K=4. For Lab: L=1, A=2, B=3. For LCH: L=1, C=2, H=3. If the data is being delivered in one of the color formats and is being delivered in "chunky" format, the **wPlane** parameter will be 0.

Chunky color data will always be delivered interleaved, in the order specified by ulColorSpace (RGB, CMYK, LAB, or LCH order), LSB aligned and byte aligned. This means that 10 bit per color data will occupy two bytes per color per pixel and the valid bits will be in the lower 10 bits of each two byte pair. The byte order is specific to the system. For the Windows environment and the Macintosh with Intel CPU, the low byte will be first.

The module can request that the data object be removed after the client is finished receiving the data by setting the **fRemoveObject** flag to TRUE. The client is not required to act on this request. If data is being delivered in more than one section, this flag should only be set to TRUE during the delivery of the final section of the data, or some data may be lost. The client may remove the data object by using the kNkMAIDCapability_Remove capability. If the current object is the only data object in the item object, the module may delete that item object. In this case, the module should send a kNkMAIDEvent_RemoveChild event to the source object.

4.25 Sound Data Delivery Structure

```
typedef struct tagNkMAIDSoundInfo
{
      NkMAIDDataInfo      base;
      ULONG               ulTotalSamples;     // number of full samples to be transferred
      BOOL                fStereo;            // TRUE if stereo, FALSE if mono
      ULONG               ulStart;            // index of starting sample of data
      ULONG               ulLength;           // number of samples of data
      UWORD               wBits;              // number of bits per channel
      UWORD               wChannel;           // 0 = mono or L+R; 1,2 = left, right
      BOOL                fRemoveObject;      // TRUE if the object should be removed
} NkMAIDSoundInfo, FAR* LPNkMAIDSoundInfo;
```

The module sets the pDataInfo parameter of the client's data delivery callback function to a pointer to this structure to describe the sound data being delivered. The **fStereo** member applies to the sound as a whole. If only one channel of a stereo sound is being transferred, it will be TRUE.

Stereo data will always be delivered interleaved, in LR order, LSB aligned and byte aligned. This means that 10 bit per channel data will occupy two bytes per channel per sample and the valid bits will be in the lower 10 bits of each two byte pair. The byte order is specific to the system. For the Windows environment and the Macintosh with Intel CPU, the low byte will be first.

The module can request that the data object be removed after the client is finished receiving the data by setting the **fRemoveObject** flag to TRUE. The client is not required to act on this request. If data is being delivered in more than one section, this flag should only be set to TRUE during the delivery of the final section of the data, or some data may be lost. The client may remove the data object by using the kNkMAIDCapability_Remove capability. If the current object is the only data object in the item object, the module may delete that item object. In this case, the module should send a kNkMAIDEvent_RemoveChild event to the source object.

4.26 Enumeration Structure

```
typedef struct tagNkMAIDEnum
{
       ULONG        ulType;             // one of eNkMAIDArrayType
       ULONG        ulElements;         // total number of elements
       ULONG        ulValue;            // current index
       ULONG        ulDefault;          // default index
       SWORD        wPhysicalBytes;     // bytes per element
       LPVOID       pData;              // allocated by the client
} NkMAIDEnum, FAR* LPNkMAIDEnum;
```

The NkMAIDEnum structure allows an enumeration to be transferred through the MAID interface. The client will always allocate the memory. It is the responsibility of the receiver of the data to interpret the data properly. The size of **pData** in bytes should always be **ulElements** times **wPhysicalBytes**.

This structure is used to implement a capability that is a choice of options. The current index will be in **ulValue** and the default index will be in **ulDefault**. If **ulType** is *kNkMAIDArrayType_String* or *kNkMAIDArrayType_PackedString*, the strings are the text representations to be presented to the user.

The value of a choice capability can be set in two ways. The client can send a pointer to an NkMAIDEnum structure with a new index in **ulValue** or it can send an unsigned integer which will be interpreted as the index.

When **ulType** is *kNkMAIDArrayType_PackedString*, **pData** will point to a packed list of null terminated strings, **ulElements** will be the total length in bytes of the data including the terminating null bytes and **wPhysicalBytes** will be one.

4.27 File Data Delivery Structure

```
typedef struct tagNkMAIDFileInfo
{
       NkMAIDDataInfo      base;
       ULONG               ulFileDataType;     // One of eNkMAIDFileDataTypes
       ULONG               ulTotalLength;      // total number of bytes to be transferred
       ULONG               ulStart;            // index of starting byte (0-based)
       ULONG               ulLength;           // number of bytes in this delivery
       BOOL                fDiskFile;          // TRUE if the file is delivered on disk
       BOOL                fRemoveObject;      // TRUE if the object should be removed
} NkMAIDFileInfo, FAR* LPNkMAIDFileInfo;
```

The module sets the pDataInfo parameter in the client's data delivery callback function to a pointer to this structure to describe the data being delivered. The ulFileDataType and ulTotalLength members apply to the file data as a whole (the type and size of the actual file). The ulStart member will contain the offset in the file of the data being delivered. The ulLength member will contain the length of the data being delivered. If the data is delivered in more than one call, the pieces should be delivered in order from beginning to end.

If the file being delivered is on disk, the fDiskFile member should be set to TRUE and the pData member of the client's data delivery callback will be set to a pointer to an NkMAIDString structure. This structure will contain the full path and name of the disk file. When delivering a file on disk, the ulStart member should be set to 0; the ulLength and ulTotalLength members should be set to the total length of the file, if it is known by the module. If the length of the file is not known by the module, these members may be set to 0. The module may not deliver data in a disk file unless the client has set the delivery location with the kNkMAIDCapability_AcceptDiskAcquisition capability first.

The module can request that the data object be removed after the client is finished receiving the data by setting the **fRemoveObject** flag to TRUE. The client is not required to act on this request. If data is being delivered in more than one section, this flag should only be set to TRUE during the delivery of the final section of the data, or some data may be lost. The client may remove the data object by using the kNkMAIDCapability_Remove capability. If the current

object is the only data object in the item object, the module may delete that item object. In this case, the module should send a kNkMAIDEvent_RemoveChild event to the source object.

4.28 LiveView Header Structure

```
typedef struct tagLiveViewHeader
{
    char            m_cLVImageCompression;
    char            m_cAFEnable;
    char            m_cAFDriveStatus;
    char            m_cFocused;
    char            m_cFaceRecognition;
    char            m_cFaceRecogNum;
    char            m_cFaceRecogIdx;
    char            m_cTrackingAFStatus;
    unsigned char   m_MovieRemainTime[4];
    char            m_MicrophoneIndicatorPeakL;
    char            m_MicrophoneIndicatorPeakR;
    char            m_MicrophoneIndicatorL;
    char            m_MicrophoneIndicatorR;
    char            m_MovieRecState;
    char            m_cHDMIRecStatus;
    char            m_cTimeCodeStatus;
    char            m_cSpotWbState;
    char            m_cRotation;
    char            m_LiveViewWBStatus;

#if defined (_WIN32) // Win
    WORD            nMajorVersion;
    WORD            nMinorVersion;
#else // Mac
    UWORD           nMajorVersion;
    UWORD           nMinorVersion;
#endif

    SIZEINFO        m_TimeRemain;
    SIZEINFO        m_TotalW;
    SIZEINFO        m_TotalH;
    SIZEINFO        m_DispAreaW;
    SIZEINFO        m_DispAreaH;
    SIZEINFO        m_DispCenterW;
    SIZEINFO        m_DispCenterH;
    SIZEINFO        m_JpegW;
    SIZEINFO        m_JpegH;
    SIZEINFO        m_AFFrameSizeW;
    SIZEINFO        m_AFFrameSizeH;
    SIZEINFO        m_AFCenterW;
    SIZEINFO        m_AFCenterH;
    SIZEINFO        m_FaceRecogAF_FrameW[AFAREASIZE];
    SIZEINFO        m_FaceRecogAF_FrameH[AFAREASIZE];
    SIZEINFO        m_FaceRecogAF_CenterX[AFAREASIZE];
    SIZEINFO        m_FaceRecogAF_CenterY[AFAREASIZE];

    CTimeCode       m_stTimeCode;

    uint32_t        LvShowInfoLength;
    uint32_t        LvImageLength;

    stSpiritLevel   m_SpiritLevel[3];

    char            m_cPictureTakingTemperatureStatus;
    char            m_cReserved1[3];
```

```
    uint32_t        m_MovieRemainTimeMSec;
    uint32_t        m_MovieElapsedTimeMec;
    char            m_cMovieElapsedTimeApplicable; //0: No 1: Yes
    char            m_cReserved2[3];
    uint32_t        m_AngleRollInteger;
    uint32_t        m_AngleRollDecimal; //Divide by 10000 to get decimal
    uint32_t        m_AnglePichInteger;
    uint32_t        m_AnglePichDecimal; //Divide by 10000 to get decimal
    uint32_t        m_AngleYawInteger;
    uint32_t        m_AngleYawDecimal; //Divide by 10000 to get decimal
}NKMAIDLiveViewHeader, FAR* LPNKMAIDLiveViewHeader;
```

4.29 LiveView Data Delivery Structure

```
typedef struct tagNkMAIDLiveViewData
{
    ULONG                   ulLvImageSize;    // Size of PIC
    UWORD                   wPhysicalBytes;
    UWORD                   wLogicalBits;
    NKMAIDLiveViewHeader    stLiveViewHeader;
    LPVOID                  pImageData;
} NkMAIDLiveViewData, FAR* LPNkMAIDLiveViewData;
```

4.30 Callback NkMAIDCSCallback Structure

```
typedef struct tagNkMAIDCSCallback
{
       LPNKFUNC   pUIReqProc;           //Type - LPMAIDUIRequestProc
       LPNKFUNC   pfnEventProc;         //Type - LPMAIDEventProc
       LPNKFUNC   pProgressProc;        //Type - LPMAIDProgressProc
       LPNKFUNC   pDataProc;            //Type - LPMAIDDataProc
       LPNKFUNC   pLiveViewDataProc;    //Type - LPLiveViewDataProc
       NKREF      refProc;
} NkMAIDCSCallback, FAR* LPNkMAIDCSCallback;
```

The client will initialize the structure by setting the pfnEventProc member (Line 2), pUIReqProc member (Line 3), pProgressProc member (Line 4), pDataProc member (Line 5), pLiveViewDataProc member (Line 6), refProc member (Line 7).

5 Result Codes

One of these values will be returned from the entry point function and sent to the completion callback function.

5.1 kNkMAIDResult_NotSupported

The module will return this value if the client attempted to perform an operation on a capability that does not exist for the specified object or if the client attempted to perform an operation that is not supported for the capability.

5.2 kNkMAIDResult_UnexpectedDataType

The module will return this value if the client sent the ulDataType parameter to the entry point function to a type that is incorrect for the command and/or capability.

5.3 kNkMAIDResult_ValueOutOfBounds

The module will return this value if the client attempts to set a capability to a value outside the allowed range for that capability.

5.4 kNkMAIDResult_BufferSize

The module will return this value in only two cases. When the client sends a *kNkMAIDCommand_GetCapInfo* command and the count does not match the number of capabilities. When the client sends a kNkMAIDCommand_CapGet for an array capability and the size specified in the array structure does not match the size of the data for that capability.

5.5 kNkMAIDResult_Aborted

The module will return this value for an asynchronous command if the client sends the *kNkMAIDCommand_Abort*, *kNkMAIDCommand_AbortToMark* or *kNkMAIDCommand_Close* command for the asynchronous command's object

5.6 kNkMAIDResult_NoMedia

The module will return this value if the client attempts to start an acquisition, autofocus, eject or some other process capability that requires some media in the device.

5.7 kNkMAIDResult_NoEventProc

The module will return this value if the client sends the *kNkMAIDCommand_EnumChildren* command without first setting the *kNkMAIDCapability_EventProc* capability to a value other than NULL.

5.8 kNkMAIDResult_NoDataProc

The module will return this value if the client starts an acquisition with *kNkMAIDCapability_Acquire* and has not specified a DataProc for the object.

5.9 kNkMAIDResult_ZombieObject

The module will return this value if the client attempts to send a command that cannot be completed because the object is no longer alive.

5.10 kNkMAIDResult_OutOfMemory

The module will return this value if some operation cannot be completed because of a low-memory condition.

5.11 kNkMAIDResult_UnexpectedError

The module will return this value if some operation cannot be completed because of an unexpected error.

5.12 kNkMAIDResult_HardwareError

The module will return this value if some operation cannot be completed because of a hardware error.

5.13 kNkMAIDResult_MissingComponent

The module will return this value if some operation cannot be completed because of a failure to find, open, or access a required file.

5.14 kNkMAIDAPIResult_DeviceNotAvailable

The module will return this error code if device is not available.

5.15 kNkMAIDAPIResult_OpenSessionFailed

The module will send this error code if session open is failed with device.

5.16 kNkMAIDAPIResult_LiveViewAlreadyStarted

The module will send this error code if receive a request to start liveview and liveview already started.

5.17 kNkMAIDAPIResult_LiveViewAlreadyStopped

The module will send this error code if receive a request to stop liveview and liveview already stopped.

5.18 kNkMAIDAPIResult_InvalidDeviceID

The module will send this error code if wrong device id passed.

5.19 kNkMAIDAPIResult_StartLiveViewFailed

The module will send this error code if LiveView start call failed.

5.20 kNkMAIDAPIResult_DeviceFailed

The module will send this error code if device is not able to process request.

5.21 kNkMAIDAPIResult_OpenSessionFailed

The module will send this error code if session open is failed with device.

5.22 kNkMAIDAPIResult_CapabilityNotSupported

The module will send this error code if an unsupported capability is called.

5.23 kNkMAIDAPIResult_OperationNotSupported

The module will send this error code if an unsupported operation is called.

5.24 kNkMAIDAPIResult_InvalidCapID

The module will send this error code if an invalid cap id is passed.

5.25 kNkMAIDAPIResult_OutOfRangeValue

The module will send this error code if value received is out of range.

5.26 kNkMAIDAPIResult_InvalidShootingType

The module will send this error code if invalid shooting type passed.

5.27 kNkMAIDAPIResult_ShootingCancelFailed

The module will send this error code if shooting cancellation is failed.

5.28 kNkMAIDAPIResult_InvalidShootCount

The module will send this error code if shoot count is not valid due to shooting not startd or wrong shooting type.

5.29 kNkMAIDAPIResult_InvalidShootingMode

The module will send this error code if shooting mode is incorrect.

5.30 kNkMAIDAPIResult_InvalidFileSavePath

The module will send this error code if file save path is not valid or exists.

5.31 kNkMAIDAPIResult_MovieStartFailed

The module will send this error code if movie recording start is failing.

5.32 kNkMAIDAPIResult_FileSaveFailed

The module will send this error code if file saving is failed.

5.33 kNkMAIDAPIResult_InvalidInterval

The module will send this error code if interval specified in interval shooting is invalid.

5.34 kNkMAIDAPIResult_InvalidStartTime

The module will send this error code if shooting start time is invalid.

5.35 kNkMAIDAPIResult_InvalidBULBExposureTime

The module will send this error code if bulb exposue time is invalid.

5.36 kNkMAIDAPIResult_InvalidArguments

The module will send this error code if invalid arguments passed in function call.

5.37 kNkMAIDAPIResult_InvalidState

The module will send this error code if calling API is not supported in current state.

5.38 kNkMAIDAPIResult_ContinousShootIntrupted

The module will send this error code if continuous shooting is interrupted.

5.39 kNkMAIDResult_NoError

The module will return this value if the command completed successfully.

5.40 kNkMAIDResult_Pending

The module will return this value if the client specified a completion callback function for a command and the module wants to return control to the client before the command is complete.

5.41 kNkMAIDResult_OrphanedChildren

The module will return this value if the client closes an object while the client still has children of that object open.

5.42 kNkMAIDResult_VendorBase

6 Events

Events are optional for the client, but not for the module. All of the conditions that the client would be notified of through the event callback function can be deduced by polling various elements.

6.1 kNkMAIDEvent_AddChild

The client can deduce this event by polling the *kNkMAIDCapability_Children* capability.

The module will send this event to the parent module, source or item object when it detects the addition of a new child source, item or data type object, respectively. If the event is sent to a module or source, the **data** parameter will be the ID of the new child. If the event is sent to an item, the **data** parameter will be one of eNkMAIDDataType.

The client can send the *kNkMAIDCommand_EnumChildren* command to request that the module enumerate all of the children of an object. The module will send a *kNkMAIDEvent_AddChild* to the object for each of the children. If there is no event callback function, the module will return the *kNkMAIDResult_NoEventProc* for the command.

6.2 kNkMAIDEvent_RemoveChild

The client can deduce this event by polling the *kNkMAIDCapability_Children* capability.

The module will send this event to the parent module, source or item object when it detects the removal of a child source, item or data type object, respectively. If the event is sent to a module or source, the **data** parameter will be the ID of the new child. If the event is sent to an item, the **data** parameter will be one of eNkMAIDDataType.

Before sending this event to the parent object, the module will first abort any asynchronous commands currently being processed for the child object and then set the child object's *kNkMAIDCapability_IsAlive* capability to FALSE.

6.3 kNkMAIDEvent_WarmingUp

The client can deduce this event by examining the *kNkMAIDCapability_WarmedUp* capability.

The module will send this event to a source object when the device enters a state where it cannot guarantee the best quality, for instance, when a light source is first turned on.

6.4 kNkMAIDEvent_WarmedUp

The client can deduce this event by examining the *kNkMAIDCapability_WarmedUp* capability.

The module will send this event to a source object when the device leaves a state where it cannot guarantee the best quality, for instance, when a light source is first turned on.

6.5 kNkMAIDEvent_CapChange

The client can deduce this event by examining the value and number of capabilities.

The module will send this event to the module, source, item, or data object when the number of capabilities or the values of any of the existing capabilities has changed. Normally, if the client sets the value of a single capability by

using the *kNkMAIDCommand_CapSet* command, this event is not required. However, if capabilities other than the one specified by the client are affected by the *kNkMAIDCommand_CapSet* command, this event should be sent.

If this event is sent to indicate a single capability change, the **data** parameter will be the ID of the capability which has changed. If this event is sent to indicate multiple capability changes or a change to the number of capabilities available, the **data** parameter will be NULL. If the values of several capabilities have changed, the module has the option of sending one CapChange event for each capability, or a single CapChange event with NULL data.

6.6 kNkMAIDEvent_OrphanedChildren

The client can deduce this event by examining the *kNkMAIDCapability_IsAlive* capability of the children objects.

The module will send this event to notify an object that is being closed that it has children objects that are still open.

6.7 kNkMAIDEvent_CapChangeValueOnly

The client can deduce this event by examining the value and number of capabilities.

The module will send this event to the module, source, item, or data object when the current value of a capabiliy has changed. This event implies that characteristics other than the current value have not changed (e.g. number of array elements, enumeration data, visibility, available operations, etc.). If any other characteristics have changed, the *kNkMAIDEvent_CapChange* event should be sent instead. Normally, if the client sets the value of a single capability by using the *kNkMAIDCommand_CapSet* command, this event is not required. However, if capabilities other than the one specified by the client are affected by the *kNkMAIDCommand_CapSet* command, this event should be sent.

If this event is sent to indicate a single capability change, the **data** parameter will be the ID of the capability which has changed. If this event is sent to indicate changes to a number of capabilities, the **data** parameter will be NULL. If the values of several capabilities have changed, the module has the option of sending one CapChangeValueOnly event for each capability, or a single CapChangeValueOnly event with NULL data.

6.8 kNkMAIDEvent_DeviceInfoChanged

The client can deduce this event by examining the kNkMAIDEvent_DeviceInfoChanged capability.The SDK will send this event if any new device connected to System, any device session open/close or device disconnected to system.

6.9 kNkMAIDEvent_ImageSaved

The client can deduce this event by examining the kNkMAIDEvent_ImageSaved capability. Once image saved on PC, SDK will send this event.

6.10 kNkMAIDEvent_ImageSavedInCard

The client can deduce this event by examining the kNkMAIDEvent_ImageSavedInCard capability. Once image saved on card, SDK will send this event.

6.11 kNkMAIDEvent_StorageFull_ImageNotSaved

The client can deduce this event by examining the kNkMAIDEvent_StorageFull_ImageNotSaved capability. Due to storage full on system,if image not saved then SDK will send this event.

6.12 kNkMAIDEvent_AcquireFailed_ImageNotSaved

The client can deduce this event by examining the kNkMAIDEvent_AcquireFailed_ImageNotSaved capability. If image is not saved due to image acquire fail from device, then SDK will send this event.

6.13 kNkMAIDEvent_ChangeSaveDestination

6.14 kNkMAIDEvent_AddImageInCard

7 Commands

For each of the commands documented here, there is an explanation of what the parameters to the MAID entry point will be.

Any command may take a significant amount of time to process. It is at the module's discretion as to whether to complete the command synchronously during one call to the module or return immediately and process the command asynchronously in another thread or during *kNkMAIDCommand_Async* commands. When the command is complete, the completion function supplied with the command will be called whether it was processed synchronously or asynchronously. The client can require that the command be processed synchronously by not specifying a completion function pointer. Asynchronous commands that are issued while the client is processing a callback (e.g. completion callback or event notification callback) may or may not be completed (at the module's discretion) until the client exits that callback.

The client may send several related asynchronous commands without waiting for previous ones to complete. The last one of the series will be the *kNkMAIDCommand_Mark* command. If one of the queued commands fails, the client can send the *kNkMAIDCommand_AbortToMark* command to abort the other commands up to and including the *kNkMAIDCommand_Mark* command. The module will call the completion function for each command with the **nResult** parameter set to *kNkMAIDResult_Aborted*.

An issue arises when the module processes some commands synchronously and others asynchronously. If a synchronous command is preceded by a command that is being processed asynchronously, the module must determine whether it is safe to process the new command immediately or not. If the module decides to delay processing the new command, the module must then decide whether to wait until after the previous command is complete and then process the new command synchronously or queue the new command to be processed asynchronously after the previous command.

For array and enumeration capabilities, two commands must be sent to get the data. To know how much memory to allocate, the client must first send the *kNkMAIDCommand_CapGet* command. The module will set all the members of the NkMAIDArray or NkMAIDEnum structure. Once the client allocates the memory and sets the **pData** member of the NkMAIDArray or NkMAIDEnum structure, that structure will be sent for the *kNkMAIDCommand_CapGetArray* command. If the size of the data changes between these two calls, the module will not store any data in the **pData** member and return *kNkMAIDResult_BufferSize*. The client should start the process over by sending another *kNkMAIDCommand_CapGet* command.

7.1 kNkMAIDCommand_Async

This command will process asynchronous commands for the specified object in a single threaded module.

pObject	May be NULL or refer to a module, source, item or data object
ulParam	Ignored
ulDataType	Ignored
data	Ignored

Multithreaded or synchronous modules can simply return *kNkMAIDResult_NoError* for this command.

If the module has the *kNkMAIDCapability_AsyncRate* capability, the client will send this command at the specified rate during idle periods.

The module will return *kNkMAIDResult_Pending* to indicate that it is processing commands and it wishes to receive this command as soon as possible and not at the rate specified by the *kNkMAIDCapability_AsyncRate* capability

7.2 kNkMAIDCommand_Open

This command will open a child of the specified object.

pObject	May be NULL or refer to a module, source or item
ulParam	May be NULL, the ID of the source or item or the type of the data object to be opened
ulDataType	Must be *kNkMAIDDataType_ObjectPtr*
data	Must be a pointer to a NkMAIDObject structure

This command will be the first command sent to a module after it is loaded. When the **pObject** parameter is NULL, the object opened will be a module object and the module will initialize itself if it has not already. When the **pObject** parameter refers to a module, source or item, the object opened will be a source, item or data object, respectively.

The client will set the **refClient** member of the NkMAIDObject structure passed in the **data** parameter before calling the module. The module will allocate any internal structures to maintain a state and store a pointer, handle, ID or other identifier in the **refModule** member of the NkMAIDObject structure. The module will set all of the NkMAIDObject structure members except **refClient**. While the object is open, the client will not change the value of the **refClient** member and the module will not change the value of the **refModule** member. No two objects can have the same value for **refClient** or **refModule**.

This command may be used more than once by the same client to open the same module, source, item or data object. The module will maintain a separate internal structure and state for each invocation.

If the command completes successfully, the client must close the object before releasing the module.

7.3 kNkMAIDCommand_Close

This command will close the connection to the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	Ignored
ulDataType	Ignored
data	Ignored

The module will abort any commands that are processing asynchronously for the object. The module will set the **refModule** member of the NkMAIDObject structure to NULL. The client may not use the structure again without reopening the object.

If the client has not closed all of the child objects of the object it is closing, the module will send the *kNkMAIDEvent_OrphanedChildren* event. If there is no event callback function or the event callback function does not close all of the child objects, the module will return *kNkMAIDResult_OrphanedChildren*.

7.4 kNkMAIDCommand_GetCapCount

This command will get the number of capabilities available for the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	Ignored
ulDataType	Must be *kNkMAIDDataType_UnsignedPtr*
data	Must be a pointer to a 32 bit unsigned integer.

7.5 kNkMAIDCommand_GetCapInfo

This command will get information about all of the capabilities available for the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	The number of NkMAIDCapInfo structures that can be stored
ulDataType	Must be *kNkMAIDDataType_CapInfoPtr*
data	Must be a pointer to an array of NkMAIDCapInfo structures

The size of the array must coincide with the **ulParam** parameter. The module will return *kNkMAIDResult_BufferSize* if it does not.

7.6 kNkMAIDCommand_CapStart

This command will start the specified capability for the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	The ID of the capability to be started
ulDataType	Ignored
data	Ignored

Within the capability's NkMAIDCapInfo structure, the **ulType** member must be *kNkMAIDCapType_Process* and the **ulOperations** member must be have the *kNkMAIDCapOperation_Start* bit set. If the capability does not support this command, the module will return *kNkMAIDResult_NotSupported*.

7.7 kNkMAIDCommand_CapSet

This command will set the value of the specified capability for the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	The ID of the capability to be set
ulDataType	One of eNkMAIDDataType
data	Value or pointer

Within the capability's NkMAIDCapInfo structure, the **ulOperations** member must be have the *kNkMAIDCapOperation_Set* bit set. The values that are permitted for **ulDataType** depends on the **ulType** member of the capability's NkMAIDCapInfo structure according to this table:

ulType value	**ulDataType** value
kNkMAIDCapType_Boolean	*kNkMAIDDataType_Boolean*, *kNkMAIDDataType_BooleanPtr*
kNkMAIDCapType_Integer	*kNkMAIDDataType_Integer*, *kNkMAIDDataType_IntegerPtr*
kNkMAIDCapType_Unsigned	*kNkMAIDDataType_Unsigned*, *kNkMAIDDataType_UnsignedPtr*
kNkMAIDCapType_Float	*kNkMAIDDataType_FloatPtr*
kNkMAIDCapType_Point	*kNkMAIDDataType_PointPtr*
kNkMAIDCapType_Size	*kNkMAIDDataType_SizePtr*
kNkMAIDCapType_Rect	*kNkMAIDDataType_RectPtr*
kNkMAIDCapType_String	*kNkMAIDDataType_StringPtr*
kNkMAIDCapType_DateTime	*kNkMAIDDataType_DateTimePtr*
kNkMAIDCapType_Callback	*kNkMAIDDataType_CallbackPtr*, *kNkMAIDDataType_Null*
kNkMAIDCapType_Array	*kNkMAIDDataType_ArrayPtr*
kNkMAIDCapType_Enum	*kNkMAIDDataType_EnumPtr*, *kNkMAIDDataType_Unsigned*
kNkMAIDCapType_Range	*kNkMAIDDataType_RangePtr*, *kNkMAIDDataType_Unsigned*
kNkMAIDCapType_Generic	*kNkMAIDDataType_GenericPtrRangePtr*

This command is not permitted for the *kNkMAIDCapType_Process* type.

If the data type does not match this table, the module will return *kNkMAIDResult_UnexpectedDataType*. If the capability does not support this command, the module will return *kNkMAIDResult_NotSupported*.

7.8 kNkMAIDCommand_CapGet

This command will get the value of the specified capability for the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	The ID of the capability to be retrieved
ulDataType	One of eNkMAIDDataType
data	Pointer

Within the capability's NkMAIDCapInfo structure, the **ulOperations** member must be have the *kNkMAIDCapOperation_Get* bit set. The values that are permitted for **ulDataType** depends on the **ulType** member of the capability's NkMAIDCapInfo structure according to this table:

ulType value	**ulDataType** value
kNkMAIDCapType_Boolean	*kNkMAIDDataType_BooleanPtr*
kNkMAIDCapType_Integer	*kNkMAIDDataType_IntegerPtr*
kNkMAIDCapType_Unsigned	*kNkMAIDDataType_UnsignedPtr*
kNkMAIDCapType_Float	*kNkMAIDDataType_FloatPtr*
kNkMAIDCapType_Point	*kNkMAIDDataType_PointPtr*
kNkMAIDCapType_Size	*kNkMAIDDataType_SizePtr*
kNkMAIDCapType_Rect	*kNkMAIDDataType_RectPtr*
kNkMAIDCapType_String	*kNkMAIDDataType_StringPtr*
kNkMAIDCapType_DateTime	*kNkMAIDDataType_DateTimePtr*
kNkMAIDCapType_Callback	*kNkMAIDDataType_CallbackPtr*
kNkMAIDCapType_Array	*kNkMAIDDataType_ArrayPtr*
kNkMAIDCapType_Enum	*kNkMAIDDataType_EnumPtr*
kNkMAIDCapType_Range	*kNkMAIDDataType_RangePtr*
kNkMAIDCapType_Generic	*kNkMAIDDataType_GenericPtr*

This command is not permitted for the *kNkMAIDCapType_Process* type.

If the data type does not match this table, the module will return *kNkMAIDResult_UnexpectedDataType*. If the capability does not support this command, the module will return *kNkMAIDResult_NotSupported*.

7.9 kNkMAIDCommand_CapGetDefault

This command will get the default value of the specified capability for the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	The ID of the capability to be retrieved
ulDataType	One of eNkMAIDDataType
data	Pointer

Within the capability's NkMAIDCapInfo structure, the **ulOperations** member must be have the *kNkMAIDCapOperation_GetDefault* bit set. The values that are permitted for **ulDataType** depends on the **ulType** member of the capability's NkMAIDCapInfo structure according to this table:

ulType value	**ulDataType** value
kNkMAIDCapType_Boolean	*kNkMAIDDataType_BooleanPtr*
kNkMAIDCapType_Integer	*kNkMAIDDataType_IntegerPtr*
kNkMAIDCapType_Unsigned	*kNkMAIDDataType_UnsignedPtr*
kNkMAIDCapType_Float	*kNkMAIDDataType_FloatPtr*
kNkMAIDCapType_Point	*kNkMAIDDataType_PointPtr*
kNkMAIDCapType_Size	*kNkMAIDDataType_SizePtr*
kNkMAIDCapType_Rect	*kNkMAIDDataType_RectPtr*
kNkMAIDCapType_Generic	*kNkMAIDDataType_GenericPtr*

This command is not permitted for the *kNkMAIDCapType_Process*, *kNkMAIDCapType_String*, *kNkMAIDCapType_DateTime*, *kNkMAIDCapType_Callback*, *kNkMAIDCapType_Array, kNkMAIDCapType_Enum,* and *kNkMAIDCapType_Range* types.

If the data type does not match this table, the module will return *kNkMAIDResult_UnexpectedDataType*. If the capability does not support this command, the module will return *kNkMAIDResult_NotSupported*.

7.10 kNkMAIDCommand_CapGetArray

This command will get the data associated with the specified array capability for the specified module, source, item or data object.

pObject	May refer to a module, source, item or data object
ulParam	The ID of the array capability for which to get data
ulDataType	Must be *kNkMAIDDataType_ArrayPtr, kNkMAIDDataType_EnumPtr*
data	Must be a pointer to an NkMAIDArray or NkMAIDEnum structure

Within the capability's NkMAIDCapInfo structure, the **ulType** member must be *kNkMAIDCapType_Array* or *kNkMAIDCapType_Enum* and the **ulOperations** member must be have the *kNkMAIDCapOperation_GetArray* bit set. All of the members of the NkMAIDArray or NkMAIDEnum structure will not be changed, the module will only store data at the address pointed to by the **pData** member. If the capability does not support this command, the module will return *kNkMAIDResult_NotSupported*. The module will return *kNkMAIDResult_BufferSize* if the members of the NkMAIDArray or NkMAIDEnum structure do not match what the module wants to store.

7.11 kNkMAIDCommand_Mark

This command will insert a mark in the queue for the specified module, source, item or data object.

pObject May refer to a module, source, item or data object
ulParam Ignored
ulDataType Ignored
data Ignored

This command is to better support asynchronous command processing by the module. The module does not need to perform any operations, but this command must not complete until all of the asynchronous commands before it for the specified object are complete. A completion function might not be supplied for this command.

7.12 kNkMAIDCommand_AbortToMark

This command will abort asynchronous commands in the queue for the specified module, source, item or data object up to and including the next *kNkMAIDCommand_Mark* command.

pObject May refer to a module, source, item or data object
ulParam Ignored
ulDataType Ignored
data Ignored

This command is to better support asynchronous command processing by the module.

The client may send several related asynchronous commands without waiting for previous ones to complete. The last one of the series will be the *kNkMAIDCommand_Mark* command. If one of the queued commands fails, the client can send the *kNkMAIDCommand_AbortToMark* command to abort the other commands up to and including the *kNkMAIDCommand_Mark* command. The module will call the completion function for each command with the **nResult** parameter set to *kNkMAIDResult_Aborted*.

If there is no *kNkMAIDCommand_Mark* command, all asynchronous commands will be aborted.

Only commands sent to the specified object will be aborted.

7.13 kNkMAIDCommand_Abort

This command will abort the asynchronous command currently being processed.

pObject May refer to a module, source, item or data object
ulParam Ignored
ulDataType Ignored
data Ignored

This command is to better support asynchronous command processing by the module. The module will call the completion function for the command with *kNkMAIDResult_Aborted*.

Only commands sent to the specified object will be aborted.

7.14 kNkMAIDCommand_EnumChildren

The module will send *kNkMAIDEvent_AddChild* events to the object for all of it's children.

pObject	May refer to a module, source or item
ulParam	Ignored
ulDataType	Ignored
data	Ignored

If the client did not set the *kNkMAIDCapability_EventProc* capability to a value other than NULL before sending this command, the module will return *kNkMAIDResult_NoEventProc*.

7.15 kNkMAIDCommand_GetParent

The module will get information about an object's parent.

pObject	May refer to a source, item or data object
ulParam	Ignored
ulDataType	Must be *kNkMAIDDataType_ObjectPtr*
data	Must be a pointer to a NkMAIDObject structure

The module will set the members of the NkMAIDObject structure to match the values of the object's parent's NkMAIDObject structure. The client may use the structure in subsequent calls to the module. It is, however, the responsibility of the client to make sure any persistent data it has for the parent object remains valid.

7.16 kNkMAIDCommand_ResetToDefault

The module will reset the selected object to its default values.

pObject	May refer to a module, source, item or data object
ulParam	Ignored
ulDataType	Ignored
data	Ignored

The module will reset all capabilities of the selected object to their default values. If the selected object has open children or data objects, those objects should also have their capabilities reset to their default values.

8 Capabilities

For each of the capabilities documented here, there is an explanation of what the members of the NkMAIDCapInfo structure will be.

Most of the capabilities listed here will be handled explicitly by the client. The remaining listed capabilities and the vendor supplied capabilities will be handled in a generic manner. The client will use the **ulVisibility** and **szDescription** members to describe them to the user.

The module has the ability to specify that a set of capabilities should be treated as a group. For each capability that is to be included in a group, the **ulVisibility** member must contain the *kNkMAIDCapVisibility_GroupMember* value. In order to group those capabilities, a new "group" capability must be created. The **ulVisibility** member must contain the *kNkMAIDCapVisibility_Group* value. This new capability will be a *kNkMAIDCapType_Array* type capability, which will contain an array of ID's of other capabilities. When the client reads this capability with the *kNkMAIDCommand_CapGet* command, the module will fill the NkMAIDArray structure as follows:

```
1      ulType = kNkMAIDArrayType_Unsigned
2      ulElements = <number of member-capabilities in this group>
3      ulDimSize1 = <same as ulElements>
4      ulDimSize2 = 0
5      ulDimSize3 = 0
6      wPhysicalBytes = 4
7      wLogicalBits = 32
8      pData = NULL
```

After allocating enough memory to hold the data, the client will call the module with the *kNkMAIDCommand_CapGetArray* command. Upon return, the pData member should contain an array of ULONG values, each one of which is the ID of another capability.

8.1 kNkMAIDCapability_AsyncRate

The module uses this capability to suggest the frequency that the client should send *kNkMAIDCommand_Async* commands during idle periods. It is expressed as the number of milliseconds (1/1000 s) between *kNkMAIDCommand_Async* commands.

Object types Module only
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get*

This capability should not be provided if a module does not require periodic *kNkMAIDCommand_Async* commands, as may be the case with a multithreaded module.

The frequency is merely a suggestion from the module to the client. The client may not be able to send the commands as fast as the module would like them.

See the description of the *kNkMAIDCommand_Async* command for more information.

8.2 kNkMAIDCapability_ProgressProc

The module will call this callback during lengthy processes.

Object types Module, source, item or data object
ulType *kNkMAIDCapType_Callback*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

If a command issued for the object will take a significant amount of time, the module will call this callback so the client can provide the user with a progress display. How often, or whether to call this callback at all, is at the module's discretion.

The initial value will be NULL. The client can indicate it does not want to be given progress information by setting this capability with the **ulDataType** parameter to the MAID entry point set to *kNkMAIDDataType_Null*.

See the description of the MAIDProgress callback function for more information.

8.3 kNkMAIDCapability_EventProc

The module will call this callback to notify the client of events.

Object types Module, source, item or data object
ulType *kNkMAIDCapType_Callback*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

The initial value will be NULL. The client can indicate it does not want to be given event notification by setting this capability with the **ulDataType** parameter to the MAID entry point set to *kNkMAIDDataType_Null*.

See the description of the MAIDEvent callback function for more information.

8.4 kNkMAIDCapability_DataProc

The module will call this callback to deliver data to the client.

Object types Data object only
ulType *kNkMAIDCapType_Callback*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

The module is required to provide this capability for data objects.

The initial value will be NULL. The client must set this capability before starting an acquire. Once the data exchange is complete, the client can set this capability with the **ulDataType** parameter to the MAID entry point set to *kNkMAIDDataType_Null*.

See the description of the MAIDData callback function for more information.

8.5 kNkMAIDCapability_UIRequestProc

The module will call this callback to request that some user interface be shown.

Object types Module only
ulType *kNkMAIDCapType_Callback*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

The module is required to provide this capability for module objects. The client must set this capability just after the module is opened. If it does not, the module might not be able to notify the user or ask the user a question.

The initial value will be NULL.

See the description of the MAIDUIRequest callback function for more information.

8.6 kNkMAIDCapability_IsAlive

This is the objects validity state.

Object types Module, source, item or data object
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get*

The module is required to provide this capability for all objects.

The value of this capability is usually TRUE. It is FALSE if the object is removed by the module or the object's parent is closed by the client.

8.7 kNkMAIDCapability_Children

This is the list of child source or item IDs.

Object types Module or source
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray*

The module is required to provide this capability for module and source objects.

Within the NkMAIDArray structure, **ulType** will be *kNkMAIDArrayType_Unsigned* and **wPhysicalBytes** will be four .

8.8 kNkMAIDCapability_State

The client can use this capability save the state of the object for later retrieval.

Object types Module, source, item or data object
ulType *kNkMAIDCapType_Array*
ulOperations *kNkMAIDCapOperation_GetArray*, *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

The data within the array is entirely module dependent and will not be interpreted by the client. The data will be saved and restored by the client verbatim.

Within the NkMAIDArray structure, **ulType** will be *kNkMAIDArrayType_Unsigned*, **wPhysicalBytes** will be one and **wLogicalBits** will be eight.

8.9 kNkMAIDCapability_Name

This is the name of the object.

Object types Module, source, item or data object
ulType *kNkMAIDCapType_String*
ulOperations *kNkMAIDCapOperation_Get*

Unlike the *kNkMAIDCapability_Description* capability, this capability cannot be set. The module should use that capability if it can store a descriptive name and it wishes to allow the user to edit that name.

8.10 kNkMAIDCapability_Description

This is the description of the object.

Object types Module, source, item or data object
ulType *kNkMAIDCapType_String* or *kNkMAIDCapType_Array*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray* for an array type, possibly *kNkMAIDCapOperation_Set*

The module will provide this capability if it can describe the object in more detail than the *kNkMAIDCapability_Name* capability. This will allow the user to better identify the object described.

The module may implement this capability as an array. The **ulType** member of the NkMAIDArray structure will be *kNkMAIDArrayType_String* or *kNkMAIDArrayType_PackedString*.

8.11 kNkMAIDCapability_Interface

This is the description of the physical interface being used to communicate with the source.

Object types Source only
ulType *kNkMAIDCapType_String*
ulOperations *kNkMAIDCapOperation_Get*

This will allow the user to better identify the source.

8.12 kNkMAIDCapability_DataTypes

This is the data types available from an item or the data types that a source can produce.

Object types Item or source
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get*

The value will be a bitwise combination of the eNkMAIDDataObjType values. The value kNkMAIDDataObjType_File should not be used in connection with this capability. This value is to be used only for data delivery.

8.13 kNkMAIDCapability_DateTime

This is the date and time of the item's capture.

Object types Item only
ulType *kNkMAIDCapType_DateTime*
ulOperations *kNkMAIDCapOperation_Get*

This capability will only be provided by modules for devices with storage capabilities. A scanner module will not provide it.

8.14 kNkMAIDCapability_StoredBytes

This is the size of the object in bytes as it is stored in the device.

Object types Item or data object
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get*

This capability will only be provided by modules for devices with storage capabilities.

8.15 kNkMAIDCapability_Eject

This will eject the media from the source device.

Object types Source or item
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

If this capability is started for a source, all media will be ejected. If it is started for an item, only the media for that item will be ejected.

8.16 kNkMAIDCapability_Feed

This will feed media into the source device.

Object types Source only
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

If there is no media to be fed, the module will return *kNkMAIDResult_NoMedia*.

8.17 kNkMAIDCapability_Capture

This will capture another item for the source device.

Object types Source only
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

Upon successful completion of this process, the source will have an addition child item. The source should enumerate it's items again.

8.18 kNkMAIDCapability_MediaPresent

Object types Data object only
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_Set*

TRUE if item has physical media to acquire.

8.19 kNkMAIDCapability_Mode

This is the acquire mode for the data object.

Object types Data object only
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray*, *kNkMAIDCapOperation_Set*

It is up to the module to decide what modes are available and what they mean. The user will make the choice from the string array. The **ulType** member of the NkMAIDEnum structure will be *kNkMAIDArrayType_String* or *kNkMAIDArrayType_PackedString*.

8.20 kNkMAIDCapability_Acquire

This will start the acquire.

Object types Data object only
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

The module will begin calling the data object's data delivery callback with data. The module may also call the data object's progress callback if the acquire will take a significant amount of time.

8.21 kNkMAIDCapability_ForceScan

This will determine whether unnecessary acquisitions (as determined by the module) will be performed by the device.

Object types Data object only
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_Set*

If this capability is set to TRUE, the device will always perform a physical scan when the *kNkMAIDCapability_Acquire* capability is started. If this capability is set to FALSE, the module may decide, based on its current state, whether such an acquisition is necessary. If a physical acquisition is not necessary, the module must go through the same steps that would normally be incurred during an acquisition (data delivery, I/O completion, etc.), except for the fact that the data would be supplied from an internal buffer as opposed to coming from a device. The default value for this capability is TRUE.

8.22 kNkMAIDCapability_Start

The starting position for the acquire in seconds.

Object types Sound or Video
ulType *kNkMAIDCapType_Float*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

This is the offset from the beginning of the sound or video object. This capability will only be provided by modules for devices with storage capabilities.

8.23 kNkMAIDCapability_Length

The length available or the length to be acquired in seconds.

Object types Sound or Video
ulType *kNkMAIDCapType_Float*
ulOperations *kNkMAIDCapOperation_Get*, possibly *kNkMAIDCapOperation_Set*, possibly *kNkMAIDCapOperation_GetDefault*

The default value will be the total length available from a module for a device with storage capabilities.

8.24 kNkMAIDCapability_SampleRate

The number of samples per second to acquire.

Object types Sound or video
ulType *kNkMAIDCapType_Enum* or *kNkMAIDCapType_Range*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray*, possibly *kNkMAIDCapOperation_Set*

Within the NkMAIDEnum structure, the **ulType** member will be *kNkMAIDArrayType_Float*.

8.25 kNkMAIDCapability_Stereo

This will select the type as either mono or stereo.

Object types Sound or video
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get*; *kNkMAIDCapOperation_Set*

If the module does not provide this capability, the client will assume that the device is only capable of mono acquires.

8.26 kNkMAIDCapability_Samples

The number of samples that will be acquired with consideration given to the current state of the data object.

Object types Sound or video
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get*

8.27 kNkMAIDCapability_Filter

This will select the filter for the light source of the device.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray*, *kNkMAIDCapOperation_Set*

Within the NkMAIDArray structure, the **ulType** member will be *kNkMAIDArrayType_Unsigned*, **wPhysicalBytes** will be four and **wLogicalBits** will be 32. The array will contain values from the eNkMAIDFilter enumeration.

8.28 kNkMAIDCapability_Prescan

The device will automatically set itself up for the original media.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

8.29 kNkMAIDCapability_ForcePrescan

This will determine whether unnecessary prescans (as determined by the module) will be performed by the device.

Object types Data object only
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_Set*

If this capability is set to TRUE, the device will always perform a physical prescan when the *kNkMAIDCapability_Prescan* capability is started. If this capability is set to FALSE, the module may decide, based on its current state, whether such a prescan is necessary. If a physical prescan is not necessary, the module must go through the same steps that would normally be incurred during a prescan (I/O completion, etc.). The default value for this capability is TRUE.

8.30 kNkMAIDCapability_ForceAutoFocus

This will determine whether unnecessary auto focus operations (as determined by the module) will be performed by the device.

Object types Data object only
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_Set*

If this capability is set to TRUE, the device will always perform a physical auto focus when the *kNkMAIDCapability_AutoFocus* capability is started. If this capability is set to FALSE, the module may decide, based on its current state, whether such an auto focus operation is necessary. If a physical auto focus is not necessary, the module must go through the same steps that would normally be incurred during an auto focus operation (I/O completion, etc.). The default value for this capability is TRUE.

8.31 kNkMAIDCapability_AutoFocus

The device will automatically set the focus of the device.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

The module should update the value of the *kNkMAIDCapability_Focus* capability if it is able to.

8.32 kNkMAIDCapability_AutoFocusPt

This is the point that the module focuses upon.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Point*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

The module is not required to provide this capability if it does not support focusing on a single point.

8.33 kNkMAIDCapability_Focus

This is the focus position of the device.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Enum* or *kNkMAIDCapType_Range*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray*, *kNkMAIDCapOperation_Set*

Within the NkMAIDEnum structure, the **ulType** member will be *kNkMAIDArrayType_Float*.

8.34 kNkMAIDCapability_Coords

This is the target area to be acquired expressed as full resolution pixels.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Rect*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*, *kNkMAIDCapOperation_GetDefault*

The default value will be the largest area than can be acquired.

8.35 kNkMAIDCapability_Resolution

This is the acquire resolution in pixels/inch.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Enum* or *kNkMAIDCapType_Range*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray*, *kNkMAIDCapOperation_Set*

Within the NkMAIDEnum structure, the **ulType** member will be *kNkMAIDArrayType_Float*.

8.36 kNkMAIDCapability_Preview

This will set a priority on speed or quality.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

If the client sets this capability to TRUE, the module should try to acquire as quickly as possible. If the client sets it to FALSE, the module should try to produce the best quality possible.

8.37 kNkMAIDCapability_Negative

This will select the type of original media as either negative or positive.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

If the module does not provide this capability, the client will make no assumptions about the original media.

8.38 kNkMAIDCapability_ColorSpace

This will select the color space of the data delivered to the client.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray*, *kNkMAIDCapOperation_Set*

Within the NkMAIDEnum structure, the **ulType** member will be *kNkMAIDArrayType_Unsigned* and **wPhysicalBytes** will be four. The enumeration will contain one or more values from the eNkMAIDColorSpace enumeration.

8.39 kNkMAIDCapability_Bits

This will select the number of bits to acquire per color.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*, *kNkMAIDCapOperation_GetArray*

If the module does not provide this capability, the client will assume that eight bits per color will be acquired.

8.40 kNkMAIDCapability_Planar

This will select or merely report the transfer mode supported by the object.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get*, possibly *kNkMAIDCapOperation_Set*

If the module only wishes to transfer the data as either planar or interleaved, it will not support the *kNkMAIDCommand_CapSet* command.

8.41 kNkMAIDCapability_Lut

This is a set of look up tables to be applied to the image data before it is transferred to the client.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Array*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*, *kNkMAIDCapOperation_GetArray*

Within the NkMAIDArray structure, the **ulType** member will be *kNkMAIDArrayType_Unsigned*. For color images, the array will be two or more look up tables, with the number of and order of the tables depending on the current color space. For RGB, there would be three tables, in the order of red, green, blue. For CMYK, there would be four tables, in the order of cyan, magenta, yellow, black. For monochrome images, there will be only one look up table.

8.42 kNkMAIDCapability_Transparency

This will select the type of original media as either transparent or reflective.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Boolean*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

If the module does not provide this capability, the client will make no assumptions about the original media.

8.43 kNkMAIDCapability_Threshold

This is the threshold level for bilevel lineart images.

Object types Image or thumbnail
ulType *kNkMAIDCapType_Range*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_Set*

8.44 kNkMAIDCapability_Pixels

The number of pixels that will be acquired with consideration given to the current state of the data object.

Object types Image, thumbnail, or video
ulType *kNkMAIDCapType_Size*
ulOperations *kNkMAIDCapOperation_Get*

8.45 kNkMAIDCapability_NegativeDefault

This is a source capability which will allow the default value of *kNkMAIDCapability_Negative* to be set.

Object types Source object only
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_Set*

Once this capability is set, the source will use that value as the default value for *kNkMAIDCapability_Negative* for all image objects opened under this source thereafter. If this capability is not supported by the source, the module will use a reasonable default value for the items.

The module may change this capability in response to a change in the hardware. If the module initiates such a change, that change must be accompanied by a *kNkMAIDEvent_CapChange* event sent to the source.

8.46 kNkMAIDCapability_Firmware

This is a source capability which reports the firmware version of a device.

Object types Source object only
ulType *kNkMAIDCapType_String*
ulOperations *kNkMAIDCapOperation_Get*

This capability allows the client to read the firmware version of a device.

8.47 kNkMAIDCapability_CommunicationLevel1

This is a source capability which will allow the client to specify the method of communication to be used with the device.

Object types Source object only
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_GetArray; kNkMAIDCapOperation_GetDefault; kNkMAIDCapOperation_Set*

The module will determine what methods of communications it can support. The user will make the choice from the string array. The **ulType** member of the NkMAIDEnum structure will be *kNkMAIDArrayType_String* or *kNkMAIDArrayType_PackedString*. One such list, for example, may include the following strings: "COM1", "COM2", "COM3", "COM4", and "SCSI". Optionally, the module may also analyze the system and eliminate any methods of communication that are not supported on that system. Using the example above, "COM3" and "COM4" may be removed if the system does not have those comm ports available.

8.48 kNkMAIDCapability_CommunicationLevel2

This is a source capability which will allow the client to specify more detail about the method of communication to be used with the device.

Object types Source object only
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_GetArray; kNkMAIDCapOperation_GetDefault; kNkMAIDCapOperation_Set*

This capability will consist of a list of strings which will further specify the method of communication to be used. The user will make the choice from the string array. The **ulType** member of the NkMAIDEnum structure will be *kNkMAIDArrayType_String* or *kNkMAIDArrayType_PackedString*. If the communication method selected in *kNkMAIDCapability_CommunicationLevel1* is "COM1", then a typical list for this capability might include the following strings: "Comm Speed 19,200", "Comm Speed 38,400", "Comm Speed 57,600", and "Comm Speed 115,200". Optionally, the module may also analyze the system and eliminate any methods of communication that are not supported on that system. If the method of communication is fully described in *kNkMAIDCapability_CommunicationLevel1* and no further information is needed, then the **ulElements** member of the NkMAIDEnum structure should be set to zero.

8.49 kNkMAIDCapability_BatteryLevel

This is a source capability which will report the level of the battery.

Object types Source object only
ulType *kNkMAIDCapType_Integer*
ulOperations *kNkMAIDCapOperation_Get*

If the device can use a battery, this capability should be supported. If the battery is in use at the time this capability is queried, the module should return an integer between 0 and 100, inclusive, which indicates the percentage of battery life remaining. If the battery is not in use at the time this capability is queried (e.g. an external power supply is attached), the module should return a value of -1.

8.50 kNkMAIDCapability_FreeBytes

This is a source capability which will report the number of bytes available in the internal memory of the device.

Object types Source object only
ulType *kNkMAIDCapType_Float*
ulOperations *kNkMAIDCapOperation_Get*

If the device can use some sort of internal storage (e.g. Compact Flash), this capability should be supported. This capability should report the number of available bytes as a positive integer value. A floating point value is used to allow for a higher upper limit.

8.51 kNkMAIDCapability_FreeItems

This is a source capability which will report the number of items that can be added to the device using the available internal memory and the current device settings.

Object types Source object only
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get*

If the device can use some sort of internal storage (e.g. Compact Flash), this capability should be supported.

8.52 kNkMAIDCapability_Remove

This capability instructs the device to remove an object from its internal memory.

Object types Source, item, data object
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

If this capability is started for an item or data object, that object should be removed from the device. If this capability is started for a source object, all item objects and their corresponding data objects should be removed from the device.

8.53 kNkMAIDCapability_FlashMode

This is the current flash mode.

Object types Source, item, data object
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get; kNkMAIDCapOperation_GetArray; kNkMAIDCapOperation_GetDefault; kNkMAIDCapOperation_Set*

Within the NkMAIDEnum structure, the **ulType** member will be *kNkMAIDArrayType_Unsigned* and **wPhysicalBytes** will be four. The enumeration will contain one or more values from the eNkMAIDFlashMode enumeration.

8.54 kNkMAIDCapability_ModuleType

This is the type of device for which this module is intended.

Object types Module object only
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get*

This capability will return one or more bit values from the eNkMAIDModuleType enumeration. This will help the client determine if this module should be used, or perhaps which user interface to display.

8.55 kNkMAIDCapability_AcquireStreamStart

This will start a stream acquire.

Object types Data object only
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

The module will begin calling the data object's data delivery callback with data. This process is designed to continue until the client stops it, so the progress callback should not be called.

8.56 kNkMAIDCapability_AcquireStreamStop

This will stop a stream acquire.

Object types Data object only
ulType *kNkMAIDCapType_Process*
ulOperations *kNkMAIDCapOperation_Start*

If a stream acquire is not in progress, the module will return *kNkMAIDResult_UnexpectedError*.

8.57 kNkMAIDCapability_AcceptDiskAcquisition

This capability is used to inform the module that it may deliver files on disk in response to *kNkMAIDCapability_Acquire*.

Object types Source object only
ulType *kNkMAIDCapType_Generic*
ulOperations *kNkMAIDCapOperation_Get, kNkMAIDCapOperation_Set*

When the client requests image data (with kNkMAIDCapability_Acquire), the module should not deliver disk files unless this capability has been called by the client with a non-null parameter. The client may use this capability to set a disk location. Once this is done, the module may deliver data by disk file, instead of by memory. After the disk file has been written and closed, the module should call the data object's data delivery callback. For Windows, the pData parameter should point to an NkMAIDString structure, which will contain the complete path (but not the name) for the new file. For Macintosh, the pData parameter should point to an FSSpec structure, which will indicate a folder for the new file. The module will choose a unique file name in the specified folder. Once the data delivery callback is called, the module should not access this file again for any reason. While preparing and writing the file to disk, the module may call the data object's progress callback if the acquisition will take a significant amount of time.

8.58 kNkMAIDCapability_Version

This is the version of the MAID specification which was followed in writing the current module.

Object types Module object only
ulType *kNkMAIDCapType_Unsigned*
ulOperations *kNkMAIDCapOperation_Get*

The client may determine the version of the MAID specification to which the module was written with this capability. This capability was introduced in MAID version 3.1. Therefore, if the module was written to a standard prior to 3.1, this capability will not be supported (will return *kNkMAIDResult_NotSupported*).

This capability will return a 4-byte unsigned value. The MAID version number will be broken into four parts, with the most significant part going into the most significant byte; the least significant part going into the least significant byte. For MAID version 3.1, for example, the most significant byte will contain 3, the next byte will contain 1, the next byte will contain 0, and the least significant byte will contain 0.

8.59 kNkMAIDCapability_FilmFormat

This will select the film format.

Object types Source object only
ulType *kNkMAIDCapType_Enum*
ulOperations *kNkMAIDCapOperation_Get*, *kNkMAIDCapOperation_GetArray, kNkMAIDCapOperation_Set*

The **ulType** member of the NkMAIDEnum structure will be *kNkMAIDArrayType_String* or *kNkMAIDArrayType_PackedString*. The enumeration will contain one or more values.

The client can select film format with this capability. For example, film format includes "35mm", "6*6" and "6*4.5". If the module supports only one format, this enumeration will contain only one value.

8.60 kNkMAIDCapability_TotalBytes

This is a source capability which will report the total number of bytes in the internal memory of the device.

Object types Source object only
ulType *kNkMAIDCapType_Float*
ulOperations *kNkMAIDCapOperation_Get*

If the device can use some sort of internal storage (e.g. Compact Flash), this capability should be supported. This capability should report the total number of bytes as a positive integer value. A floating point value is used to allow for a higher upper limit.

9 Function Definitions

9.1 MAID Entry Point Function

```
NKERROR MAIDEntryPointProc(
      LPNkMAIDObject      pObject,     // module, source, item or data object
      ULONG               ulCommand,   // Command, one of eNkMAIDCommand
      ULONG               ulParam,     // parameter for the command
      ULONG               ulDataType,  // Data type, one of eNkMAIDDataType
      NKPARAM             data,        // Pointer or long integer
      LPNKFUNC            pfnComplete, // Completion function, may be NULL
      NKREF               refComplete  // Value passed to pfnComplete
);
```

The return value will be one of eNkMAIDResult.

9.2 MAID Completion Function

```
void MAIDCompletionProc(
      LPNkMAIDObject      pObject,     // module, source, item or data object
      ULONG               ulCommand,   // Command, one of eNkMAIDCommand
      ULONG               ulParam,     // parameter for the command
      ULONG               ulDataType,  // Data type, one of eNkMAIDDataType
      NKPARAM             data,        // Pointer or long integer
      NKREF               refComplete, // Reference set by client
      NKERROR             nResult      // One of eNkMAIDResult
);
```

This is a placeholder for a callback function supplied by the client that will be called by the module after the command is complete. The parameters are the same parameters that were passed to the MAID entry point.

9.3 MAID Data Delivery Function

```
NKERROR MAIDDataProc(
      NKREF               refProc,     // Reference set by client
      LPVOID              pDataInfo,   // Cast to LPNkMAIDImageInfo or LPNkMAIDSoundInfo
      LPVOID              pData
);
```

This is a placeholder for a callback function supplied by the client that will be called by the module to deliver data. The return value will be one of eNkMAIDResult.

9.4 MAID Event Notification Function

```
void MAIDEventProc (
      NKREF               refProc,     // Reference set by client
      ULONG               ulEvent,     // One of eNkMAIDEvent
      NKPARAM             data         // Pointer or long integer
);
```

This is a placeholder for a callback function supplied by the client that will be called by the module to notify the client of events.

9.5 MAID Progress Notification Function

```
void MAIDProgressProc(
       ULONG               ulCommand,    // Command, one of eNkMAIDCommand
       ULONG               ulParam,      // parameter for the command
       NKREF               refProc,      // Reference set by client
       ULONG               ulDone,       // Numerator
       ULONG               ulTotal       // Denominator
);
```

This is a placeholder for a callback function supplied by the client that will be called by the module to notify the client of the progress of an asynchronous command.

For commands whose progress can be measured, the module will call this function at the start of the command with the **ulDone** parameter set to zero and the **ulTotal** parameter set to some positive value. When the command is complete, the module will call this function with the **ulDone** parameter equal to the **ulTotal** parameter. The module will call this function with the **ulDone** parameter set to zero and **ulTotal** exactly once each.

For commands whose progress cannot be measured, the module will call this function at the start of the command with the **ulDone** parameter set to one (1) and the **ulTotal** parameter set to zero. When the command is complete, the module will call this function with the **ulDone** parameter and the **ulTotal** parameter both equal to zero.

9.6 MAID User Interface Request Function

```
ULONG MAIDUIRequestProc(
       NKREF                       refProc,     // reference set by client
       LPNkMAIDUIRequestInfo       pUIRequest   // information about the UI request
);
```

This is a placeholder for a callback function supplied by the client that will be called by the module to notify the user or ask the user a question. The **pObject** parameter is used if the UI Request contains capabilities to display to the user. If there are no capabilities to display, this parameter may be set to NULL. The **pUIRequst** parameter will point to an NkMAIDUIRequestInfo structure, which contains information about the message, the buttons, and optionally, which capabilities to display. The return value will be one of eNkMAIDUIRequestResult.

9.7 LiveViewData Function

```
void LiveViewDataProc(
       NKREF                       ref,
       LPNkMAIDLiveViewData        pstLiveViewData
);
```

Parameters

NKREF nkRef : Reference set by GUI. It is an input parameter.
LPNkMAIDLiveViewData pstLiveViewData :

9.8 MAID EntryPointE Function

```
NKERROR MAIDEntryPointE(
      LPNkMAIDObject            pObject,
      ULONG                     ulCommand,
      ULONG                     ulParam,
      ULONG u                   lDatatype,
      NKPARAM                   data,
      LPMAIDCompletionProc      pProc,
      NKREF                     nkRef
);
```

Parameters

Return Type
NKERROR

9.9 MAID SetHinstance Function

```
BOOL SetHinstance(
      HINSTANCE          hInstDll,
      DWORD              dwReason,
      LPVOID             lpReserved
);
```

Parameters

Return Type
BOOL

9.10 MAID AllocateMemory Function

```
Void* MAIDAllocateMemory(
      size_t             memory_size
);
```

Parameters
size_t memory_size :

9.11 MAID FreeMemory Function

```
void LPMAIDFreeMemory(
      void*              memory_pointer
);
```

Parameters
void* memory_pointer :

9.12 MAID AsyncCompletion Function

```
void  MAIDAsyncCompletionProc(
       NKREF               refComplete, // Reference set by client
       NKERROR             nResult      // One of eNkMAIDResult
);
```

Parameters
NKREF refComplete: Reference set by client

NKERROR nResult : One of eNkMAIDResult

9.13 MAID InitializeSDK Function

```
NKERROR  MAIDInitializeSDK(
       LPMAIDAllocateMemory             allocMemory,
       LPMAIDFreeMemory                 freeMemory,
       LPNkMAIDCSCallback               callback,
       LPNkMAIDEnumDevices*             ppDeviceList,
       LPNkMAIDEnumCapInfo*             ppEnumCapInfo
);
```

This Simplified API is used to initialize the SDK. it will give the device list and module related capabilities.

Parameters

LPMAIDAllocateMemory allocMemory : This is an input parameter. This parameter is used to send function pointer of memory allocation API.

LPMAIDFreeMemory freeMemory : This is an input parameter. This parameter is used to send function pointer of memory free API.

LPNkMAIDCSCallback callback : This is an input parameter. Used to register callback functions. GUI need to send function pointers for different function, please refer below table.

LPNkMAIDCSCallback Member variable	Description
LPNkMAIDCSCallback.pUIReqProc	A function pointer to the UI request Proc
LPNkMAIDCSCallback.pfnEventProc	A function pointer to the event Proc
LPNkMAIDCSCallback.pProgressProc	A function pointer to the Progress Proc
LPNkMAIDCSCallback.pDataProc	A function pointer to the Data Proc
LPNkMAIDCSCallback.pLiveViewDataProc	A function pointer to the Live view data Proc

LPNkMAIDEnumDevices *ppDeviceList : This is an input/output parameter. GUI will pass the address of LPNkMAIDEnumDevices, If not null, CS Layer will allocate the required memory and copy the device List.

LPNkMAIDEnumDevices

LPNkMAIDEnumDevices Member Variables	Description
LPNkMAIDEnumDevices.ulElements	The total number of devices
LPNkMAIDEnumDevices.ulValue	Current device index
LPNkMAIDEnumDevices.pDeviceData	An object of LPNkMAIDDeviceInfo containg the individual device information.

LPNkMAIDDeviceInfo

LPNkMAIDDeviceInfo Member Variables	Description
LPNkMAIDDeviceInfo.ID	Device Id
LPNkMAIDDeviceInfo.Name	Device Name
LPNkMAIDDeviceInfo.Availability	Device id available for connection or not
LPNkMAIDDeviceInfo.ulConnectedPID	Connected process ID

LPNkMAIDEnumCapInfo *ppEnumCapInfo :This is an input/output parameter. GUI will pass the address of LPNkMAIDEnumCapInfo, If not null, CS Layer will allocate the required memory and copy the module related capabilities.

LPNkMAIDEnumCapInfo

LPNkMAIDEnumCapInfo Member Variables	Description
LPNkMAIDEnumCapInfo.pCapArray	An object of type LPNkMAIDCapInfo containing the capabilities information
LPNkMAIDEnumCapInfo.ulCapCount	Number of capability
LPNkMAIDEnumCapInfo.ulAllocationSize	Memory allocation size

LPNkMAIDCapInfo

LPNkMAIDCapInfo Member Variables	Description
LPNkMAIDCapInfo.ulID	Capability ID
LPNkMAIDCapInfo.ulType	one of eNkMAIDCapabilityType enum
LPNkMAIDCapInfo.ulVisibility	eNkMAIDCapVisibility bits. The module will set the ulVisibility member of the NkMAIDCapInfo structure to indicate what level of visibility a particular capability has.
LPNkMAIDCapInfo.ulOperations	eNkMAIDCapOperations bits. The module will set the ulOperations member of the NkMAIDCapInfo structure to indicate what operations are permitted on a particular capability
LPNkMAIDCapInfo.szDescription	text describing the capability

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When the incorrect argument is passed
kNkMAIDAPIResult_InvalidState	When the SDK state is not valid that is SDK state other than “SDK Not Loaded”.
kNkMAIDResult_OutOfMemory	When Memory allocation fail inside the SDK

9.14 MAID FreeSDK Function

```
HRESULT  MAIDFreeSDK();
```

This Simplified API is used to Free the SDK.

Parameters
None

Return Type
HRESULT

9.15 MAID ConnectDevice Function

```
NKERROR  MAIDConnectDevice(
       ULONG                          ulDeviceId,
       LPNkMAIDEnumCapInfo*           ppEnumCapInfo
);
```

This Simplified API is used to connect the Camera, Also load the source object capabilities.

Parameters

ULONG ulDeviceId : Device Id for session opening. It is an input parameter.

LPNkMAIDEnumCapInfo *ppEnumCapInfo : It is an input/output parameter. GUI will pass the address of LPNkMAIDEnumCapInfo, If not null, CS Layer will allocate the required memory and copy all device related capabilities.

LPNkMAIDEnumCapInfo Member Variables	Description
LPNkMAIDEnumCapInfo.pCapArray	It is an object of LPNkMAIDCapInfo structure. SDK fill it with device related capability array.
LPNkMAIDEnumCapInfo.ulCapCount	This member provide the information of the total number of capabilities in capability array.
LPNkMAIDEnumCapInfo.ulAllocationSize	Allocate memory size.

LPNkMAIDCapInfo

For this structure information refer section 9.13 InitializeSDK.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidDeviceID	When the incorrect device id is passed
kNkMAIDAPIResult_InvalidState	When the SDK state is not valid that is SDK state other than “SDK Loaded”.
kNkMAIDResult_OutOfMemory	When Memory allocation fail inside the SDK

9.16 MAID DisconnectDevice Function

```
HRESULT MAIDDisconnectDevice();
```

This Simplified API is used to disconnect the Camera. It will Kill live view thread and execution thread. Abort all input command if any. Clear device related capability array and start the execution thread again.

Parameters

None

Return Type
HRESULT

9.17 MAID StartLiveView Function

```
NKERROR MAIDStartLiveView(
       LPMAIDAsyncCompletionProc        pProc,
       NKREF                            nkRef
);
```

This Simplified API is used to start remote live view.

Parameters

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error

kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_StartLiveViewFailed	When unable to start live view. Some possible scenarios are Unable to get Live view status or live view status is kNkMAIDLiveViewStatus_ON_RemoteLV Unable to get CS Layer reference. Unable to get value of capability kNkMAIDCapability_LiveViewProhibit or value of kNkMAIDCapability_LiveViewProhibit is not equal to 0
kNkMAIDAPIResult_InvalidState	When the SDK state is not valid.
kNkMAIDResult_OutOfMemory	When Memory allocation fail inside the SDK

9.18 MAID StopLiveView Function

```
NKERROR MAIDStopLiveView(
       LPMAIDAsyncCompletionProc        pProc,
       NKREF                            nkRef
);
```

This Simplified API is used to stop remote live view.

Parameters

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidState	When the SDK state is not valid.

9.19 MAID GetLiveViewStatus Function

```
HRESULT MAIDGetLiveViewStatus(
       eNkMAIDLVStatus&                 LiveViewStatus
);
```

This Simplified API is used to get live view status.

Parameters
eNkMAIDLiveViewStatus &LiveViewStatus : It is an input/output parameter. SDK update the live view status, which is one of the eNkMAIDLiveViewStatus enum

Return Type
HRESULT

9.20 MAID StartShooting Function

```
NKERROR MAIDStartShooting(
      MAIDShootingStructure&          stShootParam,
      LPMAIDAsyncCompletionProc       pProc,
      NKREF                           nkRef
);
```

This Simplified API is used start shooting.

Parameters
MAIDShootingStructure &stShootParam : It is an input parameter. GUI will send information in structure required for shooting. Below are the explanation of each member of MAIDShootingStructure structure.

LPNkMAIDEnumCapInfo Member Variables	Description
MAIDShootingStructure.ShootingType	GUI pass shooting type in this member, which is one of the eNkMAIDShootingType enum. eNkMAIDShootingType.kNkSDKShootingTypeSingle = Single shooting eNkMAIDShootingType.kNkSDKShootingTypeContinuous = Continuous Shooting eNkMAIDShootingType.kNkSDKShootingTypeInterval = Interval shooting eNkMAIDShootingType.kNkSDKShootingTypeSelfTimer = Selftimer shooting eNkMAIDShootingType.kNkSDKShootingTypeBULB = Bulb Shooting
MAIDShootingStructure.ulContinuous_Interval_NumShots	GUI will provide n umber of shots in case of interval shooting or continuous shooting.
MAIDShootingStructure.ulBULBExposureDuration	GUI will provide Bulb exposure time in second when the shooting type is set to bulb.
MAIDShootingStructure.ulShootingStartTimeFromNow	GUI will provide Start time in second when the shooting type is self-timer or interval shooting.
MAIDShootingStructure.ulIntervalTime	GUI will provide the interval time seconds when the shooting type is interval shooting.\
MAIDShootingStructure.bAutoFocus	GUI will provide information either auto focus is true or false.
MAIDShootingStructure.ImageSavePath	GUI will provide the directory path where the image should be save. Default path can also be set by **SetImageVideoSavePath** simplified API refer 3.21

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidState	When the SDK state is not valid.
kNkMAIDAPIResult_InvalidArguments	When the incorrect argument is passed
kNkMAIDAPIResult_InvalidShootingType	When the Shooting type is invalid.
kNkMAIDAPIResult_InvalidShootingMode	When the shooting mode is invalid.
kNkMAIDAPIResult_InvalidBULBExposureTime	When BULBExposureTime is <= minimum time(0).
kNkMAIDAPIResult_InvalidInterval	When interval time in case of interval shooting is 0.
kNkMAIDAPIResult_InvalidShootCount	When Shoot count is less then 2 in case of interval shooting.

9.21 MAID CancelShooting Function

```
NKERROR MAIDCancelShooting(
       LPMAIDAsyncCompletionProc        pProc,
       NKREF                            nkRef
);
```

This Simplified API is used cancel shooting.

Parameters
LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is not null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_ShootingCancelFailed	When cancel shooting failed.
kNkMAIDAPIResult_InvalidShootingType	When the Shooting type is invalid.

9.22 MAID GetShootingStatus Function

```
HRESULT MAIDGetShootingStatus(
      eNkMAIDShootingStatus&          ShootingStatus
);
```

This Simplified API is used get shooting status.

Parameters
eNkMAIDShootingStatus &ShootingStatus : It is an input/output parameter, SDK update the shooting status, which is one of the eNkMAIDShootingStatus enum.

Return Type
HRESULT

9.23 MAID GetRemainingShootCount Function

```
HRESULT MAIDGetRemainingShootCount(
      ULONG&                          RemShootCount
);
```

This Simplified API is used get remaining shoot count in case of continuous shooting and interval shooting.

Parameters
ULONG &RemShootCount : It is an input/output parameter, SDK update the remaining shoot count.

Return Type
HRESULT

9.24 MAID GetCapability Function

```
NKERROR MAIDGetCapability(
      ULONG                           ulCapabilityId,
      eNkSDKGetSettingRequestType     eType,
      LPVOID&                         pDataPtr,
      eNkMAIDDataType&                ulDataType
);
```

This Simplified API is used get the capability value.

Parameters
ULONG ulCapabilityId : Capability id to get the value.

eNkSDKGetSettingRequestType eType: One of the value of eNkSDKGetSettingRequestType enum.

eNkSDKGetSettingRequestType Member Variables	Description
eNkSDKGetSettingRequestType.kNkSDKGetSettingValue	For getting the current value of the given capability.
eNkSDKGetSettingRequestType.kNkSDKGetSettingSupportedValueArray	For getting all supported values array that can be get or set for the given capability.
eNkSDKGetSettingRequestType.kNkSDKGetSettingDefaultValue	For getting the default value of given capability.

LPVOID &pDataPtr : It is an input/output parameter, SDK allocate the required memory based on the data type of capability and copy the data.

eNkMAIDDataType &ulDataType : It is an input/output parameter, GUI will provide data type of capability.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.
kNkMAIDAPIResult_OperationNotSupported	When the requested operation not supported.
kNkMAIDAPIResult_CapabilityNotSupported	When requested capability not supported.
kNkMAIDResult_UnexpectedDataType	When an unexpected data type found for he given capability.
kNkMAIDAPIResult_InvalidState	When SDK not in valid state for getting capability values.
kNkMAIDResult_OutOfMemory	When memory allocation fail inside the SDK.

Other than above error Some other errors result code can also return based on the specific capability. For that refer that capability description.

9.25 MAID SetCapability Function

```
NKERROR MAIDSetCapability(
       ULONG                          ulCapabilityId,
       LPVOID                         pData,
       eNkMAIDDataType                ulDataType
);
```

This Simplified API is used set the capability value.

Parameters
ULONG ulCapabilityId : Capability id to set the value.

LPVOID &pDataPtr : It is an input parameter, *pData is being new value of capability.

eNkMAIDDataType &ulDataType : It is an input/output parameter, ulDataType to determine data type of pData.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.

kNkMAIDAPIResult_OperationNotSupported	When the requested operation not supported.
kNkMAIDAPIResult_CapabilityNotSupported	When requested capability not supported.
kNkMAIDResult_UnexpectedDataType	When an unexpected data type found for he given capability.
kNkMAIDAPIResult_InvalidState	When SDK not in valid state for getting capability values.
kNkMAIDResult_OutOfMemory	When memory allocation fail inside the SDK.

Other than above error Some other errors result code can also return based on the specific capability. For that refer that capability description.

9.26 MAID EnumCapabilities Function

```
NKERROR MAIDEnumCapabilities(
       LPNkMAIDEnumCapInfo&             pEnumCapInfo,
       LPMAIDAsyncCompletionProc        pProc,
       NKREF                            nkRef
);
```

This Simplified API is used get the all capability values and capability count.

Parameters
LPNkMAIDEnumCapInfo &pEnumCapInfo : pEnumCapInfo to get all the capability information with capability count. SDK will allocate required memory and copy the data.

For LPNkMAIDEnumCapInfo structure member detail please refer 9.13 InitializeSDK

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is not null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.
kNkMAIDAPIResult_InvalidState	When SDK not in valid state for getting capability values.

9.27 MAID EnumDevices Function

```
NKERROR MAIDEnumDevices(
       LPNkMAIDEnumDevices&              pEnumDeviceInfo,
       LPMAIDAsyncCompletionProc         pProc,
       NKREF                             nkRef
);
```

This Simplified API is used get the device list and availability of the devices.

Parameters
LPNkMAIDEnumDevices &pEnumDeviceInfo : pEnumDeviceInfo to get all the device information with device count and availability info. CS Layer will allocate required memory and copy the data.For detail of

LPNkMAIDEnumDevices please refer section 9.13 InitializeSDK

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is not null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.
kNkMAIDAPIResult_InvalidState	When SDK not in valid state for getting capability values.

9.28 MAID GetLoggingLevel Function

```
NKERROR MAIDGetLoggingLevel(
       eNkMAIDLoggingLevel&              LoggingLevel
);
```

This Simplified API is used get current log level.

Parameters
eNkMAIDLoggingLevel &LoggingLevel : It is an input/output parameter, SDK fill one of the value of eNkMAIDLoggingLevel enum.

Return Type
HRESULT

9.29 MAID SetLoggingLevel Function

```
HRESULT MAIDSetLoggingLevel (
      eNkMAIDLoggingLevel              LoggingLevel
);
```

This Simplified API is used set log level.

Parameters
eNkMAIDLoggingLevel LoggingLevel : It is an input parameter, SDK set one of the value of eNkMAIDLoggingLevel enum.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDResult_ValueOutOfBounds	When incorrect log level is passed.

9.30 MAID StartMovieRecording Function

```
NKERROR MAIDStartMovieRecording (
      LPSTR                            sPath,
      LPMAIDAsyncCompletionProc        pProc,
      NKREF                            nkRef
);
```

This Simplified API is used start movie recording.

Parameters
LPSTR sPath : Path to save video file.

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is not null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.
kNkMAIDResult_OutOfMemory	When memory allocation failed inside the SDK.

9.31 MAID StopMovieRecording Function

```
NKERROR MAIDStopMovieRecording (
       LPMAIDAsyncCompletionProc        pProc,
       NKREF                            nkRef
);
```

This Simplified API is used stop movie recording.

Parameters
LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is not null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.
kNkMAIDResult_OutOfMemory	When memory allocation failed inside the SDK.

9.32 MAID StartOperation Function

```
NKERROR MAIDStartOperation (
       ULONG                            ulCapabilityId,
       LPMAIDAsyncCompletionProc        pProc,
       NKREF                            nkRef
);
```

This Simplified API is used start operation for some capabilities.

Parameters
ULONG ulCapabilityId : Capability Id to start the operation.

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is not null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.

9.33 MAID MarkAsynCallSequence Function

```
NKERROR MAIDMarkAsynCallSequence ();
```

Return Type
NKERROR

9.34 MAID AbortAsynCallOrSequence Function

```
NKERROR MAIDAbortAsynCallOrSequence (
       ULONG                              ulCapabilityId,
       LPMAIDAsyncCompletionProc          pProc,
       NKREF                              nkRef
);
```

Parameters
BOOL AbortToMark

LPMAIDAsyncCompletionProc pProc : It is an input/output parameter, To judge Api call is sync or async. If pProc is not null means Async."pProc" will be called after completion of Api.

NKREF nkRef : Reference set by GUI. It is an input parameter.

Return Type
NKERROR

9.35 MAID SetImageVideoSavePath Function

```
For Windows
NKERROR MAIDSetImageVideoSavePath (
       const wchar_t*             pImageSavePath,
       const wchar_t*             pVideoSavePath
);

For macOS
NKERROR MAIDSetImageVideoSavePath (
       const char*                pImageSavePath,
       const char*                pVideoSavePath
);
```

This Simplified API is used start operation for some capabilities.

Parameters
const wchar_t *pImageSavePath : Folder path to save images. For Mac it is char* instead of wide char.
const wchar_t *pVideoSavePath : Folder path to save video. For Mac it is char* instead of wide char.

Return Type
NKERROR

Result Code	Description
kNkMAIDResult_NoError	No error
kNkMAIDResult_UnexpectedError	When an unexpected error is occur
kNkMAIDAPIResult_InvalidArguments	When incorrect argument is passed.

kNkMAIDResult_OutOfMemory	When memory allocation failed inside the SDK.

9.36 MAID IsDeviceFullSupport Function

```
BOOL MAIDIsDeviceFullSupport();
```

Return Type
BOOL

10 History

10.1 New